COORDENADORIA DE ENGENHARIA DE COMPUTAÇÃO

BRENO DE PÁDUA SOARES

GUILHERME CAMBI MAGAROTTI

GUILHERME GROSSI DE ALMEIDA

MARKETPLACE CONTEXTUAL ADAPTATIVO: UMA ENGINE DE SERVER-DRIVEN UI (SDUI) PARA INCENTIVO À LOGÍSTICA VERDE

SOROCABA, SP
2026

BRENO DE PÁDUA SOARES

GUILHERME CAMBI MAGAROTTI

GUILHERME GROSSI DE ALMEIDA

MARKETPLACE CONTEXTUAL ADAPTATIVO: UMA ENGINE DE SERVER-DRIVEN UI (SDUI) PARA INCENTIVO À LOGÍSTICA VERDE

Trabalho de conclusão de curso apresentado ao Centro Universitário Facens como exigência parcial para obtenção do diploma de graduação em Engenharia de Computação.
Orientador (a): Marco Antonio Montebello Júnior.

SOROCABA, SP

2026

LISTA DE ILUSTRAÇÕES

Figura 1 – Projeção de crescimento do comércio eletrônico no Brasil 6
Figura 2 – Gráfico comparando as emissões entre carro e caminhão 7
Figura 3 – Fluxograma geral da metodologia 12
Figura 4 – Arquitetura da solução proposta 14
Figura 5 – Página inicial no modo de inspeção SDUI 24
Figura 6 – Tela de detalhe do produto: selo com o cálculo aberto, mapa de origem e comparador de ofertas 27
Figura 7 – Checkout simulado: resumo, modalidades de entrega, remessas e sugestão de troca 28
Figura 8 – Arquitetura implementada 33
Figura 9 – Modelo de dados simplificado 34
Figura 10 – Fluxo de composição de uma tela 38
Figura 11 – Universos de dados das análises e da aplicação 42
Figura 12 – Distância em linha reta e distância de rota 44
Figura 13 – Oferta local, compra local e CO_2 evitado por UF: (a) compradores com oferta local disponível e compras feitas a menos de 100 km; (b) CO_2 evitado nos itens com alternativa 46
Figura 14 – Categoria destacada por UF e intensidade do sinal regional 48
Figura 15 – Página inicial com contexto regional (SP) e sem sinal regional (AM) 49
Figura 16 – Latência p95 em função da vazão, por estado do cache 51
Figura 17 – Evolução temporal da latência nos primeiros 5 minutos, por estado do cache (mediana de três repetições) 52
Figura 18 – Mutações do contrato SDUI na tela de detalhe: (a) original; (b) blocos reordenados; (c) selo injetado; (d) bloco novo; (e) bloco desconhecido 54
Figura 19 – Blocos da tela de detalhe reordenados pelo próprio servidor no modo de apresentação 55

LISTA DE SIGLAS

API	Application Programming Interface
BFF	Backend-for-Frontend
CEP	Código de Endereçamento Postal
CO_2e	Dióxido de carbono equivalente
ETL	Extract, Transform, Load
GEE	Gases de efeito estufa
GLEC	Global Logistics Emissions Council
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
MAE	Mean Absolute Error (erro absoluto médio)
MAPE	Mean Absolute Percentage Error (erro percentual absoluto médio)
ODS	Objetivos de Desenvolvimento Sustentável
OSRM	Open Source Routing Machine
RPS	Requisições por segundo
SDUI	Server-Driven UI
TTFB	Time to First Byte (tempo até o primeiro byte)
UF	Unidade Federativa
UI	User Interface
UX	User Experience
VU	Virtual User (usuário virtual)

RESUMO

O Marketplace Contextual Adaptativo é um protótipo de comércio eletrônico construído sobre o conjunto de dados público da Olist, no qual o servidor define a interface por meio de uma engine de Server-Driven UI (SDUI). Um Backend-for-Frontend (BFF) em FastAPI, organizado em fatias verticais, com PostgreSQL e cache de respostas em Redis, envia ao cliente uma árvore de componentes tipados que ele apenas renderiza. Sobre essa arquitetura, a vitrine se adapta à região do comprador, derivada do CEP, e cada oferta exibe distância e pegada de carbono estimadas por um motor baseado em atividade, alinhado à ISO 14083 e ao GLEC Framework, com selo para entregas a menos de 100 km. Corrigida por fator de circuidade, a distância estimada apresentou erro de 8,7% em relação a rotas do OSRM. No conjunto completo da Olist, escolher o vendedor mais próximo reduziria em 9,8% a emissão estimada das compras que tinham alternativa, ou 0,9% do total, com efeito concentrado nas regiões com mais vendedores. Com 1.000 usuários virtuais, o cache elevou a vazão de 153 para cerca de 400 requisições por segundo e reduziu a latência mediana de 4,1 s para 95 ms, embora o percentil 95 tenha ficado em torno de 3 s. Testes de mutação do contrato mostraram que a interface reflete alterações da árvore enviada pelo servidor sem nova compilação do cliente. Os resultados indicam a viabilidade técnica da abordagem, dentro dos limites de um protótipo com dados históricos e emissões estimadas.

Palavras-chave: Server-Driven UI. Logística Verde. Backend-for-Frontend. Marketplace Contextual. FastAPI.

ABSTRACT

The Adaptive Contextual Marketplace is an e-commerce prototype built on the public Olist dataset, in which the server defines the interface through a Server-Driven UI (SDUI) engine. A Backend-for-Frontend (BFF) in FastAPI, organized in vertical slices, with PostgreSQL and a Redis response cache, sends the client a tree of typed components that the client only renders. On top of this architecture, the storefront adapts to the buyer's region, derived from the postal code, and each offer shows distance and carbon footprint estimated by an activity-based engine aligned with ISO 14083 and the GLEC Framework, with a badge for deliveries under 100 km. Corrected by a circuity factor, the estimated distance showed an 8.7% error against OSRM routes. In the full Olist dataset, choosing the nearest seller would reduce the estimated emissions of purchases that had an alternative by 9.8%, or 0.9% of the total, with the effect concentrated in regions with more sellers. With 1,000 virtual users, the cache raised throughput from 153 to about 400 requests per second and reduced median latency from 4.1 s to 95 ms, although the 95th percentile stayed around 3 s. Contract mutation tests showed that the interface reflects changes in the server-sent tree without rebuilding the client. The results indicate the technical feasibility of the approach, within the limits of a prototype with historical data and estimated emissions.

Key words: Server-Driven UI. Green Logistics. Backend-for-Frontend. Contextual Marketplace. FastAPI.

SUMÁRIO

1 INTRODUÇÃO 4
2 CRESCIMENTO DO COMÉRCIO ELETRÔNICO NO BRASIL 6
2.1 SUSTENTABILIDADE E LOGÍSTICA VERDE NO E-COMMERCE 7
2.2 ARQUITETURAS MODERNAS PARA INTERFACES DINÂMICAS 8
2.2.1 *Server-Driven UI* (SDUI) 8
2.2.2 *Backend-for-Frontend* (BFF) 9
2.2.3 *Micro-frontends* 9
2.3 CÁLCULO DA PEGADA DE CARBONO NA LOGÍSTICA 9
2.4 TECNOLOGIAS APLICADAS À PERSONALIZAÇÃO DE INTERFACES 10
3 METODOLOGIA 12
3.1 TRATAMENTO E EXPLORAÇÃO DO DATASET PÚBLICO DA OLIST 13
3.2 MODELAGEM DA ARQUITETURA DE SOFTWARE E FLUXOS DE DADOS 14
3.3 CONFIGURAÇÃO DOS TESTES E DAS MÉTRICAS DE DESEMPENHO 18
4 DESENVOLVIMENTO 23
4.1 ESPECIFICAÇÃO 23
4.1.1 Requisitos funcionais 23
4.1.2 Requisitos não funcionais, métricas e critérios de aceitação 29
4.1.3 Delimitações do escopo 31
4.2 PROJETO 32
4.2.1 Organização em Vertical Slices 32
4.2.2 Modelo de dados relacional (PostgreSQL) 33
4.2.3 Contrato SDUI (esquema e tipos de componentes) 35
4.3 IMPLEMENTAÇÃO 36
4.3.1 Tratamento e carga do conjunto de dados Olist 37
4.3.2 BFF em FastAPI: motor de cálculo logístico e cache (Redis) 37
4.3.3 Composição das telas e renderização no cliente 39
4.3.4 Ambiente de execução (Docker Compose) 39
5 RESULTADOS 41
5.1 RESULTADOS QUANTITATIVOS 41
5.1.1 Qualidade e cobertura dos dados 42
5.1.2 Verificação funcional do motor de cálculo 43
5.1.3 Acurácia da distância: Haversine e OSRM 43

5.1.4 Potencial estimado de redução de emissões 45
5.1.5 Personalização contextual regional 47
5.1.6 Desempenho sob carga e efeito do cache 49
5.2 RESULTADOS QUALITATIVO-ESTRUTURAIS 53
5.2.1 Desacoplamento por mutação do contrato 53
5.2.2 Robustez do contrato: componentes desconhecidos e dados insuficientes 55
5.3 SÍNTESE DOS RESULTADOS 56
6 DISCUSSÃO E CONSIDERAÇÕES FINAIS 58
6.1 DISCUSSÃO 58
6.1.1 Relação de compromisso no projeto 58
6.1.2 Aderência aos objetivos e critérios de aceitação 60
6.2 LIMITAÇÕES E AMEAÇAS À VALIDADE 62
6.3 CONCLUSÃO 63
6.4 TRABALHOS FUTUROS 64
REFERÊNCIAS 66

1 INTRODUÇÃO

Este estudo apresenta o cenário de crescimento acelerado do comércio eletrônico brasileiro e os desafios técnicos e ambientais que acompanham essa expansão. O aumento progressivo do consumo digital exige não apenas interfaces que se adaptem rapidamente a diferentes contextos, mas também a adoção de práticas sustentáveis na logística de entrega.

Com a expansão do *e-commerce*, a manutenção das interfaces enfrenta gargalos provenientes da rigidez do código no lado do cliente, demandando ciclos frequentes de atualização e publicação. Paralelamente, surge a necessidade de uma logística verde como resposta aos impactos ambientais gerados pelo transporte de mercadorias, como as emissões de gases de efeito estufa.

Com plataformas como a Olist democratizando o acesso ao mercado para pequenos comerciantes, surge a problemática central: projetar uma infraestrutura de *software* capaz de orquestrar interfaces de forma dinâmica, eliminando a dependência de atualizações no *front-end* e, simultaneamente, incentivando escolhas logísticas de menor impacto ambiental.

O objetivo geral deste trabalho é desenvolver uma *engine* de *Server-Driven UI* (SDUI) que utilize o *dataset* real da Olist para fornecer personalização contextual da interface de forma automatizada e adaptativa. Para atingir este propósito, definiram-se os seguintes objetivos específicos:

a) projetar uma arquitetura SDUI orientada a contexto, eliminando a dependência de atualizações constantes no *front-end*;

b) implementar uma *engine* de cálculo de pegada de carbono logística, utilizando a metodologia baseada em atividade (*Activity-Based*);

c) integrar a personalização da interface com dados logísticos, para identificar compras locais e destacar alternativas de menor impacto ambiental;

d) avaliar o desempenho e a flexibilidade da solução proposta, validando a capacidade da infraestrutura em suportar a demanda exigida.

Considerando que micro e pequenas empresas brasileiras enfrentam barreiras de competitividade, o tema mostra-se não apenas atual, mas também altamente pertinente ao cenário tecnológico nacional. A justificativa fundamenta-se nos Objetivos de Desenvolvimento Sustentável (ODS) 8, 9 e 12 da ONU, promovendo a inovação

industrial, o crescimento econômico e o incentivo ao consumo responsável. O setor de *e-commerce* e a sociedade precisam aliar inovação tecnológica com responsabilidade ambiental para a resolução desta questão.

Para desenvolver o presente estudo, adotou-se a *Vertical Slice Architecture* e o padrão *Backend-for-Frontend* (BFF). O projeto utiliza o *dataset* público da Olist, composto por mais de 100 mil pedidos, que servirá como base de dados real para validação da *engine* e dos mecanismos de personalização. O *back-end* será implementado em FastAPI, enquanto o *front-end* em React atuará como um renderizador genérico. A validação técnica será realizada por meio de testes de carga com a ferramenta Locust, visando mensurar a latência e a vazão sob estresse, com o uso de *cache* em Redis.

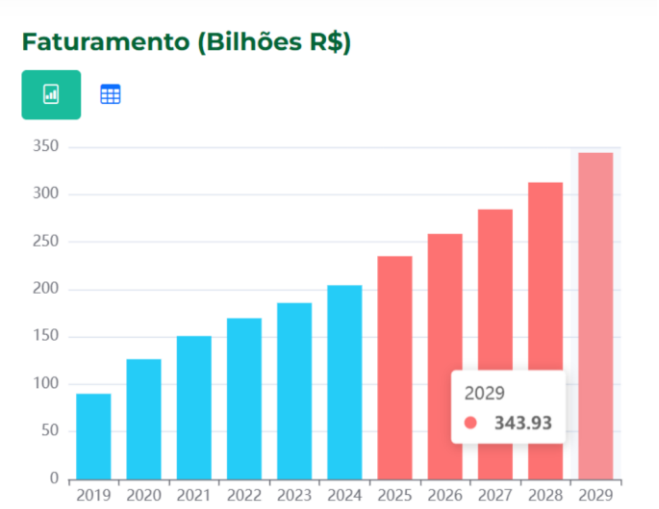
Além da aplicação prática, a metodologia inclui um levantamento bibliográfico embasado em fontes recentes (2022 a 2026). Autores de relevância na área de arquitetura de *software*, como Rocha (2022), com seus estudos sobre microsserviços orientados a eventos, e Rappl (2024), abordando a escalabilidade de interfaces dinâmicas e *micro-frontends*, ofereceram fundamentação teórica relevante para o desenvolvimento da *engine*. Essa pesquisa, aliada a publicações institucionais da ONU Brasil, visa à exposição de argumentos técnicos e sustentáveis que propiciam um bom entendimento de toda a infraestrutura proposta.

2 CRESCIMENTO DO COMÉRCIO ELETRÔNICO NO BRASIL

Este capítulo apresenta o panorama do crescimento do comércio eletrônico no Brasil e suas implicações nas operações logísticas e nas arquiteturas de *software*. Inicialmente, aborda-se a expansão do *e-commerce* e a necessidade emergente de práticas de sustentabilidade por meio da logística verde. Em seguida, discutem-se arquiteturas modernas para interfaces dinâmicas, essenciais para suportar essa demanda. Por fim, detalha-se o cálculo da pegada de carbono na logística e as tecnologias aplicadas à personalização dessas plataformas.

O crescimento exponencial do comércio eletrônico no Brasil exigiu que as empresas reavaliassem seus modelos de operação e a forma como interagem com os consumidores. Conforme ilustra a Figura 1, essa expansão não se limitou apenas ao volume de vendas, mas refletiu-se também em um aumento significativo da complexidade operacional. Para lidar com essa demanda, tornou-se necessária a adoção de plataformas de vendas que proporcionassem uma experiência de usuário mais fluida e personalizada (Oliveira, 2022).

Figura 1 – Projeção de crescimento do comércio eletrônico no Brasil



Fonte: Edrone. Disponível em: https://edrone.me. Acesso em: 14 abr. 2026

A consolidação do comércio eletrônico como um pilar da economia moderna é impulsionada pela integração de tecnologias inteligentes e pela evolução das expectativas dos consumidores em relação à agilidade e confiança nos processos

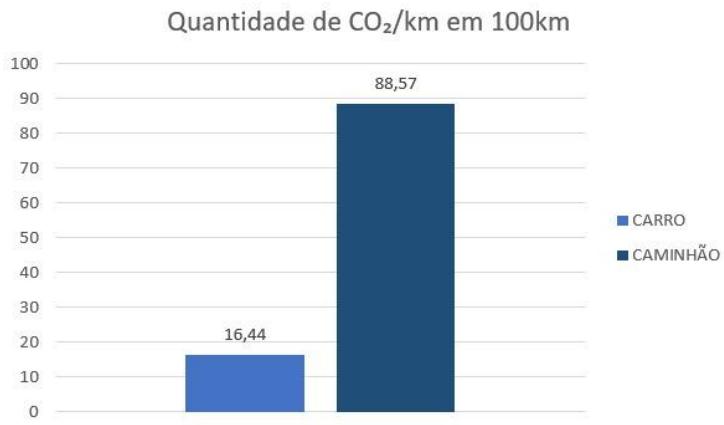
digitais (Zubaidah et al., 2024). No cenário nacional, essa maturidade reflete tendências que exigem das organizações uma adaptação constante às flutuações de mercado e à crescente competitividade no setor varejista (Oliveira, 2022). Essa complexidade operacional e o aumento expressivo no volume de acessos simultâneos demandam infraestruturas de *software* altamente escaláveis e robustas, capazes de oferecer respostas rápidas e garantir a estabilidade do sistema frente às constantes mudanças do mercado (Rocha, 2022).

2.1 SUSTENTABILIDADE E LOGÍSTICA VERDE NO E-COMMERCE

Acompanhando o crescimento da economia digital brasileira e mundial, o impacto ambiental gerado pela logística de distribuição de produtos tornou-se uma das maiores preocupações nas discussões sobre o futuro do varejo. Uma resposta direta a esse problema é a adoção da logística verde, uma estratégia voltada para a mitigação dos efeitos negativos da distribuição, como a emissão de gases de efeito estufa (GEE) durante o transporte de mercadorias (Pereira, 2024).

No Brasil, onde a matriz de transporte de cargas é predominantemente rodoviária, a otimização de rotas e dos modais de entrega ganha extrema importância. A Figura 2 demonstra que os veículos tradicionais utilizados para o transporte de cargas apresentam altos níveis de emissão de CO_2, evidenciando a urgência da implementação de práticas operacionais mais sustentáveis.

Figura 2 – Gráfico comparando as emissões entre carro e caminhão



Fonte: Elaborada pelos autores (2026) com base em Transvias (2025).

A disponibilização de plataformas digitais voltadas ao monitoramento de indicadores de sustentabilidade assegura a transparência necessária para orientar o comportamento do usuário durante a jornada de compra (Simão; Salles; Fernandes, 2024). Sob essa ótica, a criação de infraestruturas de serviço inteligentes voltadas ao suporte da sociedade permite que a mensuração do impacto ecológico de cada remessa atue como o pilar inicial para o controle rigoroso das operações, resultando na mitigação das emissões e na consolidação de uma logística integrada e sustentável (Silva, 2023).

2.2 ARQUITETURAS MODERNAS PARA INTERFACES DINÂMICAS

Considerando que as interfaces de *e-commerce* devem evoluir e permanecer flexíveis frente a demandas sazonais e ao comportamento variável dos usuários, o desenvolvimento de *software* passou a exigir arquiteturas conscientes do contexto (*context-aware*) que superem as restrições dos modelos tradicionais e inflexíveis de *front-end* (Nawaz et al., 2022).

2.2.1 *Server-Driven UI* (SDUI)

No modelo tradicional, o cliente dita o visual e o comportamento da interface do usuário. A arquitetura *Server-Driven UI* (SDUI), por sua vez, propõe uma inversão de responsabilidades alinhada aos princípios de microsserviços no *front-end*, permitindo que o servidor controle a estrutura da interface (Perlin et al., 2023). Dessa forma, ao ajustar elementos em arquivos JSON, torna-se viável implementar alterações visuais e dinâmicas na aplicação sem a necessidade de submeter atualizações constantes nas lojas de aplicativos. Em sistemas de *e-commerce*, o uso de SDUI permite que vitrines e fluxos de finalização de compras sejam dinâmicos e customizáveis com base em regras de negócio. Ao separar a lógica de renderização da definição do componente, obtém-se maior agilidade operacional, permitindo que a equipe de *back-end* controle a experiência do usuário (UX) de forma centralizada e escalável.

2.2.2 *Backend-for-Frontend* (BFF)

O padrão *Backend-for-Frontend* (BFF) atua como uma camada intermediária dedicada a atender as necessidades específicas da interface do usuário (Rattanukul et al., 2023). Para agregar e formatar dados relevantes, o *front-end* interage com um único ponto de entrada (o BFF), em vez de consumir diretamente múltiplos microsserviços genéricos. No contexto da integração entre o cálculo de carbono e a *engine* de interface, o BFF desempenha um papel fundamental, garantindo que as informações cheguem ao cliente de forma eficiente e com menor custo computacional. Essa arquitetura reduz o processamento no *client-side* e simplifica a manutenção, uma vez que as regras de composição da interface residem em um ambiente controlado no servidor.

2.2.3 *Micro-frontends*

No ecossistema de desenvolvimento *front-end*, a abordagem de *micro-frontends* consiste em dividir a aplicação em áreas menores, que são construídas, mantidas e alocadas de forma independente (Rappl, 2024). Essa técnica evita que o sistema como um todo seja comprometido por falhas isoladas, além de facilitar a colaboração entre equipes e a evolução tecnológica de componentes específicos, lidando com os desafios de implementação em larga escala (Cheripurathu; Kulkarni, 2024). Integrados a uma estrutura de SDUI, os *micro-frontends* podem ser utilizados para hospedar componentes complexos, como calculadoras de impacto ambiental e mapas interativos de rotas. Isso garante um código limpo e eficaz, permitindo que sistemas robustos manipulem grandes bases de dados para aprimorar suas funcionalidades.

2.3 CÁLCULO DA PEGADA DE CARBONO NA LOGÍSTICA

A quantidade de emissão de CO_2 na jornada de entrega é calculada tendo como base a distância aproximada entre o centro de distribuição e o destino final, além de considerar o peso da mercadoria e o fator de emissão médio do combustível

utilizado. Este cálculo considera um modelo de otimização bi-objetivo com vertentes ambientais e econômicas (Mundim, 2024).

A partir de modelos matemáticos específicos, cruza-se dados geográficos e operacionais para gerar estimativas e indicadores de impacto ambiental. Para mapear o fluxo de mercadorias no território nacional e aplicar fórmulas que estimem a eficiência ecológica de rotas alternativas, utiliza-se o *Brazilian E-Commerce Public Dataset* (Olist, 2024). Contudo, cabe ressaltar as limitações inerentes ao modelo proposto: por se tratar de uma projeção, o cálculo não contempla variáveis dinâmicas de roteamento, tais como condições de tráfego em tempo real, topografia das vias, perfil de condução do motorista ou particularidades mecânicas dos veículos. Além disso, a dependência de dados baseados em códigos postais (CEPs) restringe a granularidade exata do trajeto trecho a trecho.

Apesar destas restrições, este motor de cálculo integrado cumpre seu propósito arquitetural ao permitir que o sistema identifique rotas viáveis e sinalize ao usuário aquelas com maior potencial ecológico. Dessa forma, contribui para a conscientização sobre o desperdício de recursos energéticos e para a redução da pegada de carbono, alinhando a logística a análises de sustentabilidade (Cano et al., 2022).

Por fim, pode-se compreender que os modelos de cálculo da pegada de carbono possibilitam estimar o impacto ambiental das operações logísticas a partir de variáveis mensuráveis, embora apresentem limitações inerentes ao uso de fatores médios. Ainda assim, quando utilizados como estimativas informativas, esses modelos tornam-se relevantes para apoiar a tomada de decisão do usuário, especialmente quando integrados dinamicamente à interface por meio de uma engine orientada por dados, como a proposta neste trabalho.

2.4 TECNOLOGIAS APLICADAS À PERSONALIZAÇÃO DE INTERFACES

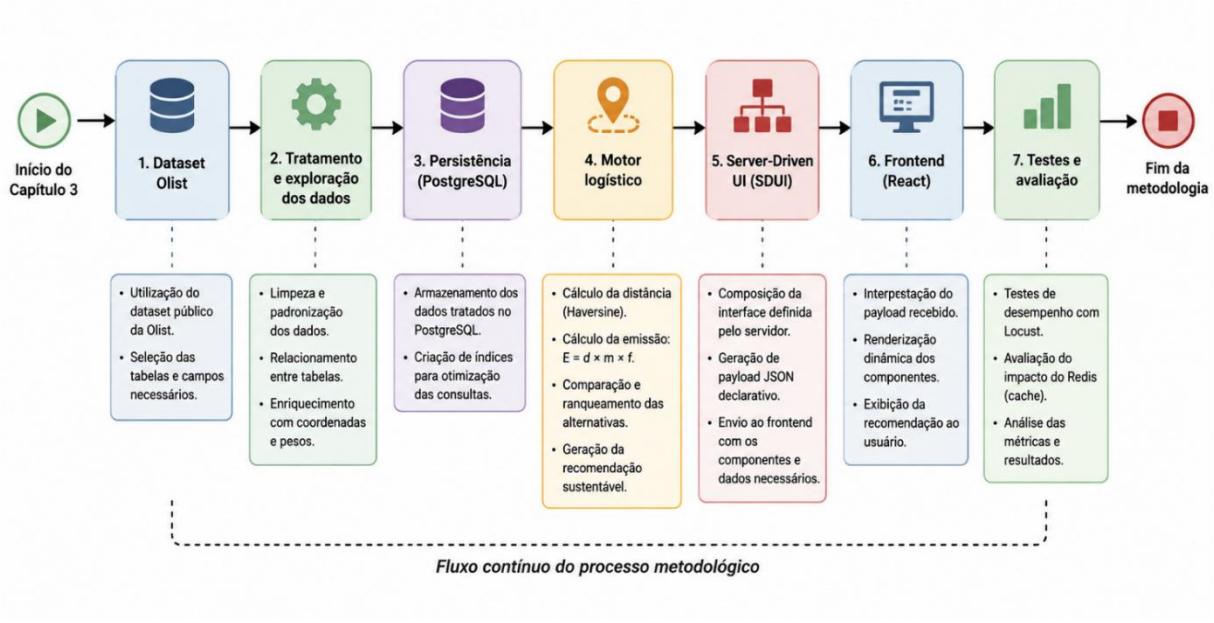
Para sustentar uma infraestrutura baseada em SDUI com cálculos executados em tempo real, é imperativo o uso de um conjunto tecnológico de alto desempenho, voltado para a construção de arquiteturas altamente escaláveis e orientadas a eventos. Por essa razão, adota-se o padrão *Backend-for-Frontend* (BFF), que

centraliza a agregação das regras de negócio e formata os dados conforme a necessidade da interface (RATTANUKUL et al., 2023). Nesse sentido, o *framework* FastAPI é utilizado no *backend* para combinar alta velocidade de execução, tipagem estrita com Pydantic e processamento assíncrono nativo (RAMÍREZ, 2024). No *frontend*, a biblioteca React oferece um vasto ecossistema para a criação de interfaces, facilitando a renderização guiada pelo servidor. O uso de banco de dados em memória, como o Redis, atua no processo de *cache*, isto é, de retenção transitória das definições de interface e dos resultados dos cálculos, assegurando escalabilidade e baixa latência, além de mitigar acessos repetitivos ao banco relacional (REDIS, 2024). Por fim, para evitar pontos de falha e atestar a estabilidade da plataforma sob estresse, o sistema é validado em simulações de picos de demanda por meio da ferramenta de testes de carga Locust.

3 METODOLOGIA

Este capítulo descreve os procedimentos empregados na preparação dos dados, na modelagem da arquitetura de software e no planejamento da avaliação técnica da solução, de modo que será dedicado à fundamentação conceitual, etapa que apresenta como o projeto será executado, quais componentes serão utilizados e de que maneira os resultados serão obtidos e analisados. O percurso metodológico foi dividido em três etapas: tratamento e exploração do dataset público da Olist; modelagem da arquitetura e dos fluxos de dados; e configuração do ambiente de testes e das métricas de desempenho, como visto na Figura 3.

Figura 3 – Fluxograma geral da metodologia



Fonte: Elaborada pelos autores (2026).

Quanto à natureza, o trabalho caracteriza-se como uma pesquisa aplicada, pois busca desenvolver uma solução computacional para integrar personalização dinâmica de interfaces e recomendações logísticas de menor impacto ambiental. A abordagem será predominantemente quantitativa, considerando o tratamento de registros de comércio eletrônico, o cálculo estimado de emissões e a análise de métricas de desempenho. Também serão empregadas observações qualitativas para verificar a flexibilidade da interface orientada pelo servidor.

3.1 TRATAMENTO E EXPLORAÇÃO DO DATASET PÚBLICO DA OLIST

A base de dados utilizada será o Brazilian E-Commerce Public Dataset by Olist, disponibilizado publicamente na plataforma Kaggle. O conjunto contém informações anonimizadas sobre mais de cem mil pedidos realizados em diferentes *marketplaces* brasileiros e reúne dados de pedidos, itens, produtos, vendedores, clientes, pagamentos, avaliações e localização geográfica (OLIST, 2024). Para este trabalho, serão selecionadas somente as tabelas relacionadas ao cálculo logístico e à composição contextual da interface.

Inicialmente, os arquivos no formato CSV serão importados em Python com a biblioteca Pandas. A exploração inicial compreenderá a identificação das dimensões de cada tabela, a inspeção dos tipos de dados, a análise das chaves de relacionamento, a contagem de valores ausentes e a identificação de registros duplicados. Essa etapa permitirá verificar se os campos necessários ao cálculo, como prefixos de CEP, latitude, longitude, peso do produto e identificação do vendedor, apresentam qualidade suficiente para utilização no motor de recomendações.

A integração será realizada principalmente entre as tabelas de pedidos, itens de pedido, produtos, vendedores, clientes e geolocalização. A tabela de itens permitirá relacionar cada pedido ao vendedor e ao produto; a tabela de produtos fornecerá o peso da mercadoria; as tabelas de clientes e vendedores fornecerão os prefixos de CEP; e a tabela de geolocalização permitirá associar esses prefixos às respectivas coordenadas geográficas. Os relacionamentos serão executados por meio das chaves disponibilizadas no próprio conjunto, mantendo-se apenas os campos necessários às etapas seguintes.

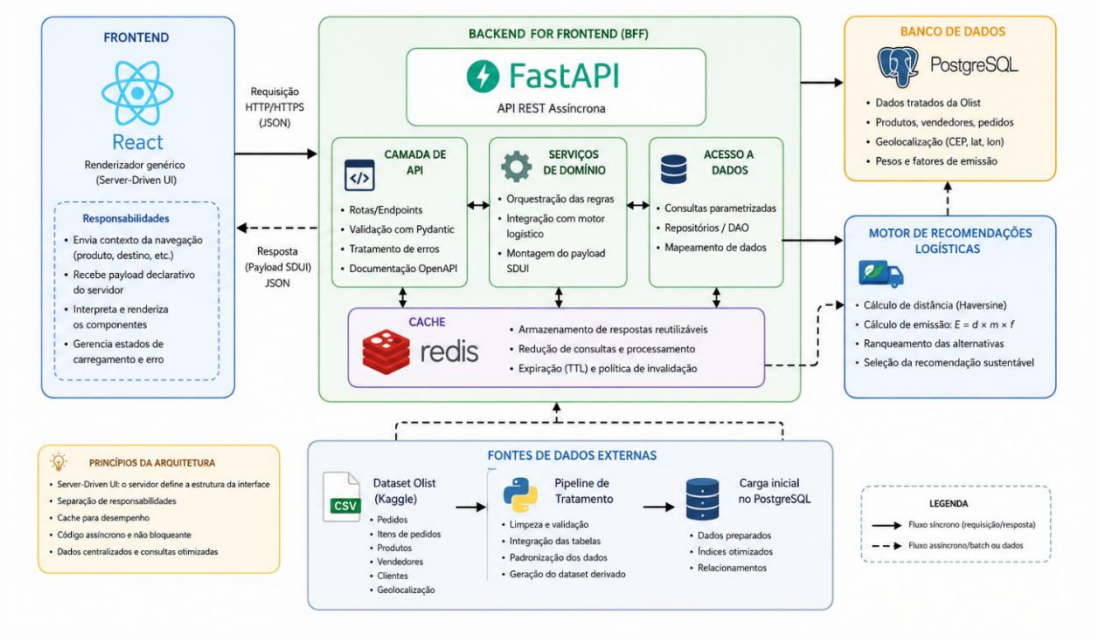
O processo de higienização incluirá a remoção de duplicidades, a padronização dos tipos de dados e o tratamento dos registros sem informações essenciais. Pedidos sem coordenadas válidas de origem ou destino e produtos sem peso utilizável não poderão alimentar o cálculo da emissão. Para prefixos de CEP associados a múltiplas coordenadas, será calculada uma coordenada representativa, preferencialmente pela mediana da latitude e da longitude, reduzindo a influência de valores discrepantes. Também serão verificadas coordenadas fora dos limites geográficos esperados para o território brasileiro e valores de peso iguais ou inferiores a zero.

Os arquivos originais serão preservados sem alterações. Ao término da etapa de transformação, será gerado um conjunto derivado e reproduzível, contendo a identificação do pedido, vendedor e produto, os prefixos de CEP, as coordenadas de origem e destino, o peso padronizado e os campos necessários ao cálculo de distância e emissão. Esse conjunto será utilizado pelo serviço de recomendação e armazenado no PostgreSQL para consultas posteriores.

3.2 MODELAGEM DA ARQUITETURA DE SOFTWARE E FLUXOS DE DADOS

A arquitetura proposta será composta por uma interface web em React, uma camada Backend-for-Frontend implementada em FastAPI, um banco relacional PostgreSQL e uma camada de cache em Redis. O React funcionará como renderizador genérico, enquanto o servidor será responsável por consultar os dados, executar as regras de negócio, calcular o impacto logístico e determinar a estrutura da interface enviada ao usuário. Essa divisão preserva o princípio da Server-Driven UI, no qual as decisões de composição permanecem centralizadas no servidor (PERLIN et al., 2023). A organização desses componentes e suas próprias interações é mostrado na Figura 4.

Figura 4 – Arquitetura da solução proposta



Fonte: Elaborada pelos autores (2026).

O fluxo será iniciado quando o usuário acessar uma página de produto ou solicitar opções de vendedores. O *front-end* enviará ao BFF as informações de contexto necessárias, incluindo o produto selecionado e o destino da entrega. O BFF validará a requisição e consultará o PostgreSQL para recuperar os vendedores disponíveis, a localização de origem, o peso do produto e os demais campos logísticos. Antes de realizar um novo processamento, o serviço verificará se existe uma resposta válida no Redis para a combinação consultada.

Quando não houver resposta armazenada em cache, o BFF acionará o motor de recomendações logísticas. A adoção dessa fórmula fundamenta-se na complexidade analítica constante $(O(1))$ e na estabilidade numérica para representações em ponto flutuante, viabilizando o processamento dinâmico no servidor sem onerar o tempo de resposta da interface. As diretrizes internacionais de mensuração de emissões em transporte de carga estabelecem que, na ausência de rastreamento veicular contínuo, a distância ortodrômica calculada em linha reta pode ser utilizada como estimativa inicial de referência, desde que documentada como uma aproximação e auditada frente à malha viária real (SMART FREIGHT CENTRE, 2023; ISO, 2023).

A distância será calculada pela Equação 1:

$$d = 2R\ arcsin\left(\sqrt{\left\{sin^{2\left(\frac{\{\varphi_2-\varphi_1\}}{2}\right)} + cos(\varphi_1)cos(\varphi_2)sin^{2\left(\frac{\{\lambda_2-\lambda_1\}}{2}\right)}\right\}}\right)$$

Em que d representa a distância geodésica em quilômetros, R corresponde ao raio médio da Terra, considerado igual a 6.371 km, φ_1 e φ_2 representam as latitudes dos pontos de origem e destino, em radianos; e λ_1 e λ_2 correspondem às respectivas longitudes, também em radianos.

É importante destacar que a distância obtida pela fórmula de Haversine não corresponde necessariamente ao percurso realizado pelas rodovias, pois não considera curvas, desvios, restrições viárias, pedágios ou condições de tráfego. Portanto, neste trabalho, o resultado será empregado como uma aproximação geográfica padronizada para comparar vendedores submetidos ao mesmo método de cálculo, e não como uma representação exata da distância efetivamente percorrida pelo veículo. Essa abordagem está alinhada a diretrizes internacionais de logística sustentável, que preveem o uso de distâncias geográficas de referência na ausência de rotas detalhadas (ISO, 2023).

Como forma de avaliar a influência dessa simplificação, será selecionada uma amostra de 100 pares de origem e destino, distribuída entre diferentes faixas de distância. Para cada par, o resultado de Haversine será comparado com a distância de uma rota calculada pelo *Open Source Routing Machine* (OSRM), serviço baseado na malha viária do *OpenStreetMap*. A comparação será realizada por meio do erro absoluto médio (MAE), do erro percentual absoluto médio (MAPE) e do coeficiente de correlação de Spearman. Também será verificado o percentual de casos em que os dois métodos preservam a mesma ordenação dos vendedores.

A utilização dessas métricas permitirá verificar não somente a diferença entre as distâncias calculadas, mas também se a aproximação de Haversine é capaz de manter a finalidade principal do sistema: identificar comparativamente as alternativas de menor impacto ambiental. Caso sejam constatadas divergências relevantes, as limitações do método serão apresentadas na análise dos resultados, e as distâncias fornecidas pelo sistema serão classificadas explicitamente como estimativas comparativas.

Após a obtenção da distância, a pegada de carbono da entrega é calculada pela abordagem *Activity-Based* (baseada na atividade de transporte), expressa pela relação:

$$E = d \times m \times f$$

Em que E representa a emissão estimada em kg CO_2, d corresponde à distância estimada em quilômetros (km), m representa a massa transportada convertida para toneladas (t), e f corresponde ao fator de emissão do transporte rodoviário, expresso em kg $CO_2/(\text{t} \cdot \text{km})$. Em conformidade com a ferramenta oficial do Programa Brasileiro GHG Protocol (2026), adotou-se o fator $f = 0{,}104$ kg $CO_2/(\text{t} \cdot \text{km})$, valor de referência para a frota rodoviária média brasileira de veículos de carga a diesel. Essa parametrização assegura a consistência dimensional da equação e fornece um indicador auditável para classificar os vendedores com menor impacto ambiental estimado.

O fator de emissão representa a quantidade estimada de gases de efeito estufa liberada durante o transporte de uma tonelada de carga ao longo de um quilômetro. Sua utilização permite relacionar a distância percorrida e a massa transportada ao impacto ambiental estimado da entrega. Para garantir a consistência dimensional do cálculo, todas as grandezas serão convertidas para unidades compatíveis antes da aplicação da equação.

A estimativa obtida não representa uma medição direta das emissões do veículo, mas um indicador calculado a partir de dados logísticos e de parâmetros representativos do transporte rodoviário. Dessa forma, o resultado será utilizado para comparar as alternativas disponíveis sob os mesmos critérios, permitindo identificar aquelas com menor impacto ambiental estimado.

Após calcular as alternativas válidas, o motor ordenará os vendedores de acordo com o impacto estimado. A opção de menor emissão será utilizada como referência sustentável, e a redução relativa poderá ser apresentada em comparação com outra alternativa disponível.

Com base no resultado logístico e nas regras de contexto, a *engine* SDUI montará um *payload* JSON contendo os componentes a serem renderizados. A resposta poderá incluir o tipo do componente, textos, valores de emissão, percentual

estimado de redução, selo de recomendação e ações disponíveis. O *React* interpretará esse contrato e associará cada tipo recebido ao componente visual correspondente, sem executar as regras responsáveis pela recomendação.

Para evitar falhas na interface, o contrato de comunicação deverá validar os campos obrigatórios e prever um componente padrão para tipos desconhecidos. Quando não houver informações suficientes para o cálculo, o servidor não exibirá uma recomendação ambiental, evitando a apresentação de resultados baseados em dados incompletos ou inconsistentes.

Respostas reutilizáveis serão armazenadas temporariamente no Redis, diminuindo consultas repetitivas ao PostgreSQL e reprocessamentos do motor logístico. Dessa forma, requisições equivalentes poderão utilizar resultados previamente calculados, contribuindo para a redução do tempo de resposta da aplicação.

3.3 CONFIGURAÇÃO DOS TESTES E DAS MÉTRICAS DE DESEMPENHO

O ambiente de desenvolvimento será organizado com Docker e Docker Compose, permitindo executar os serviços de forma isolada e reproduzível. A composição deverá incluir, conforme o estágio de implementação, os contêineres do *back-end* FastAPI, PostgreSQL, Redis e *front-end* React. O código-fonte será mantido em um repositório Git, e as dependências serão registradas em arquivos próprios de cada serviço, facilitando a reprodução do ambiente utilizado nos testes.

Antes dos testes de desempenho, serão realizados testes funcionais com entradas controladas. Serão selecionados casos em que distância, massa e fator de emissão possuam valores previamente conhecidos, permitindo conferir a conversão das unidades e o resultado da equação. Também serão verificadas a ordenação das alternativas, a geração do *payload*, a ausência de recomendação quando os dados forem insuficientes e a renderização dos componentes esperados pelo React.

Os testes de desempenho e estresse serão executados utilizando a ferramenta Locust, adotando-se o padrão de carga em degraus (*step-load pattern*) para simular o comportamento concorrente de usuários na interface web. A carga será aplicada de forma progressiva sobre o *endpoint* responsável pela orquestração da SDUI e pelo cálculo de pegada de carbono, intercalando requisições com um tempo de reflexão

(*think time*) parametrizado entre um e três segundos, modelando a navegação humana no catálogo de produtos.

Para determinar a capacidade do sistema e os limites de saturação, foram delineados três cenários de teste com dimensões e objetivos operacionais distintos:

a) Cenário de Carga Nominal (Linha de Base): simulação com 50 usuários virtuais simultâneos (*Virtual Users* - VU), inicializados a uma taxa de 5 usuários por segundo (*spawn rate* = 5 VU/s), sustentados em regime permanente por 5 minutos, visando estabelecer o tempo de resposta e a vazão em condições normais de uso;

b) Cenário de Carga Operacional Típica: aumento para 250 usuários simultâneos com taxa de inicialização de 10 usuários por segundo (*spawn rate* = 10 VU/s), mantidos por 10 minutos, com o objetivo de analisar a estabilidade do *pool* de conexões assíncronas do FastAPI e do PostgreSQL;

c) Cenário de Estresse e Saturação (Pico): dimensionado com 1.000 usuários virtuais e taxa de injeção acelerada de 25 usuários por segundo (*spawn rate* = 25 VU/s), mantidos em pico por 15 minutos, para identificar o ponto de esgotamento da CPU, a formação de filas no servidor e a taxa de erros sob condições extremas.

A Tabela 1 sintetiza o planejamento dimensional e a parametrização dos cenários de teste de carga.

Tabela 1 – Parâmetros de dimensionamento dos cenários de teste no Locust

Parâmetro	**Cenário 1 (Nominal)**	**Cenário 2 (Operacional)**	**Cenário 3 (Estresse)**
Usuários Virtuais Concorrentes (VU)	50	250	1.000
Taxa de Inicialização (*Spawn Rate*)	5 VU/s	10 VU/s	25 VU/s

Parâmetro	Cenário 1 (Nominal)	Cenário 2 (Operacional)	Cenário 3 (Estresse)
Tempo de *Ramp-up* (subida)	10 s	25 s	40 s
Duração em Regime (*Plateau*)	5 min	10 min	15 min
Think Time do Usuário	1 a 3 s	1 a 3 s	1 a 3 s
Repetições por Condição de Cache	3 vezes	3 vezes	3 vezes

Fonte: Elaborada pelos autores (2026).

O protocolo experimental adotará procedimentos rigorosos de isolamento de variáveis para neutralizar interferências transientes do sistema operacional e oscilações no agendamento de processos em segundo plano. Cada um dos cenários dimensionados será executado em triplicata (três ensaios independentes) sob três estados da camada de persistência transitória em Redis: *cache* desabilitado, *cache* frio (*cold cache*) e *cache* aquecido (*warm cache*). No primeiro estado (*bypass* integral), o sistema é forçado a percorrer todo o ciclo computacional síncrono e assíncrono, que compreende a consulta relacional no PostgreSQL, o cálculo trigonométrico geodésico de Haversine e a montagem dinâmica da árvore de nós da SDUI, sendo que irá fixar a linha de base (*baseline*) de esforço da CPU e de saturação de conexões. No estado de *cache* frio, a persistência em memória é inicializada vazia para mensurar a penalidade de processamento decorrente de rajadas simultâneas de falhas de localização de chaves (*cache misses*). Por fim, o *cache* aquecido é condicionado por meio de um ciclo prévio de requisições de carga (*pre-warming*), viabilizando avaliar a capacidade limite de vazão da camada BFF quando os contratos JSON já residem estruturados em memória rápida. Para suprimir ruídos decorrentes de atrasos

dinâmicos de rede externa (*jitter* e perda de pacotes), os contêineres do motor de aplicação, da base de dados e dos nós de injeção de carga do Locust serão orquestrados sob uma mesma sub-rede virtualizada isolada em modo *bridge*, garantindo o controle estrito do ambiente e a reprodutibilidade dos ensaios.

Quanto à avaliação do tempo de resposta, a metodologia descarta o emprego exclusivo da média aritmética como indicador primário de desempenho. Em plataformas baseadas em laço de eventos assíncronos e operações de I/O não bloqueantes, como o FastAPI, a distribuição dos tempos de resposta é tipicamente assimétrica; nesses cenários, a média mascara comportamentos patológicos que degradam a experiência de parcelas expressivas de usuários. Conforme apontam Perlin et al. (2023), a validação de arquiteturas assíncronas de baixa latência voltadas à orquestração de interfaces e serviços exige a inspeção da latência mediana ($p50$) conjugada aos percentis de cauda ($p95$ e $p99$), os quais representam o teto do tempo de espera suportado por 95% e 99% das chamadas atendidas, respectivamente. Esse acompanhamento em cauda longa é determinante para identificar gargalos provocados pela concorrência no *event loop*, pelo bloqueio transitório do *pool* de conexões e por contenções na desserialização de dados complexos. Em complemento, será traçada a correlação entre a vazão em requisições por segundo (RPS) e a curva de elevação dos percentis de latência, com a finalidade de mapear o ponto de inflexão (*knee of the curve*), que é o estágio operacional em que o enfileiramento de requisições passa a degradar os tempos de resposta antes da perda efetiva de pacotes.

A estabilidade e a integridade da arquitetura serão quantificadas pela taxa percentual de requisições com falha, segregadas segundo a classificação dos códigos de resposta do protocolo HTTP. Diante de picos de concorrência que sobrecarreguem o intermediador BFF ou esgotem os descritores de arquivos e *sockets* do sistema operacional, o comportamento de degradação será auditado por meio do registro de respostas com erro de processamento interno (HTTP 500) e indisponibilidade ou tempo limite de comunicação esgotado (HTTP 503 e 504), conforme a especificação do motor de testes de carga distribuídos (LOCUST, 2024). Os dados brutos gerados pelas três iterações de cada cenário serão submetidos a tratamento estatístico descritivo, calculando-se a média e o respectivo desvio padrão para cada métrica

monitorada. Essa consolidação quantitativa garantirá a solidez analítica das conclusões, permitindo mensurar com precisão a redução da latência e o ganho de escalabilidade proporcionados pelo desacoplamento via SDUI e pela retenção de respostas em *cache* distribuído.

Os resultados quantitativos obtidos nos ensaios de estresse serão sintetizados em tabelas comparativas e gráficos de dispersão e evolução temporal, confrontando a vazão em requisições por segundo com a elevação das curvas de latência nos percentis $p50$, $p95$ e $p99$. Para garantir a auditabilidade e a reprodutibilidade integral dos experimentos, cada conjunto de dados será catalogado com os metadados do ambiente de execução, compreendendo o *hash* do *commit* no repositório Git, as versões das imagens dos contêineres Docker, o volume consolidado de dados persistido no PostgreSQL e o arquivo de parametrização (*locustfile*) empregado nos testes.

De forma complementar à dimensão quantitativa, a flexibilidade da arquitetura Server-Driven UI será avaliada por meio de uma verificação qualitativa-estrutural de desacoplamento. Esse procedimento consistirá na execução de testes de mutação de contrato no servidor BFF, alterando dinamicamente a hierarquia dos nós da árvore JSON, como a reordenação de blocos de conteúdo, a injeção condicional de selos ecológicos de logística verde e a inserção de novos componentes visuais sem alterações no código-fonte do cliente web. A validação do desacoplamento será atestada pela capacidade do renderizador genérico em React de interpretar e refletir as novas diretrizes visuais em tempo de execução (*runtime*), sem demandar recompilação de pacotes (*build*) ou intervenções estruturais na camada de apresentação.

4 DESENVOLVIMENTO

Este capítulo apresenta o desenvolvimento do marketplace contextual adaptativo proposto neste trabalho. A solução foi construída a partir do conjunto de dados público da Olist e combina uma interface orientada pelo servidor com mecanismos de personalização contextual e de apoio à logística verde. A exposição está organizada em três momentos. Primeiro, são apresentados os requisitos e as delimitações que orientaram o desenvolvimento. Em seguida, descrevem-se a arquitetura, o modelo de dados e o contrato de comunicação entre o servidor e a interface. Por fim, são detalhadas as principais decisões de implementação, incluindo o tratamento dos dados, o cálculo logístico, a composição das telas e o ambiente de execução.

O objetivo dessa organização é explicar como os componentes se relacionam e de que maneira cada decisão contribui para os objetivos definidos no trabalho. O artefato descrito neste capítulo é o mesmo que foi submetido aos testes funcionais, à avaliação do contrato de interface e ao protocolo de carga apresentados no Capítulo 5.

4.1 ESPECIFICAÇÃO

O desenvolvimento foi iniciado pela definição do que o sistema deveria realizar e das condições que orientariam sua avaliação. Nesta etapa, foram relacionados os objetivos do trabalho às funcionalidades da jornada de compra, às exigências de desempenho e às limitações impostas pelo conjunto de dados da Olist. A especificação, portanto, não foi tratada apenas como uma lista de funções, mas como a delimitação do comportamento esperado e dos critérios que permitiriam verificar se a solução atendia à proposta.

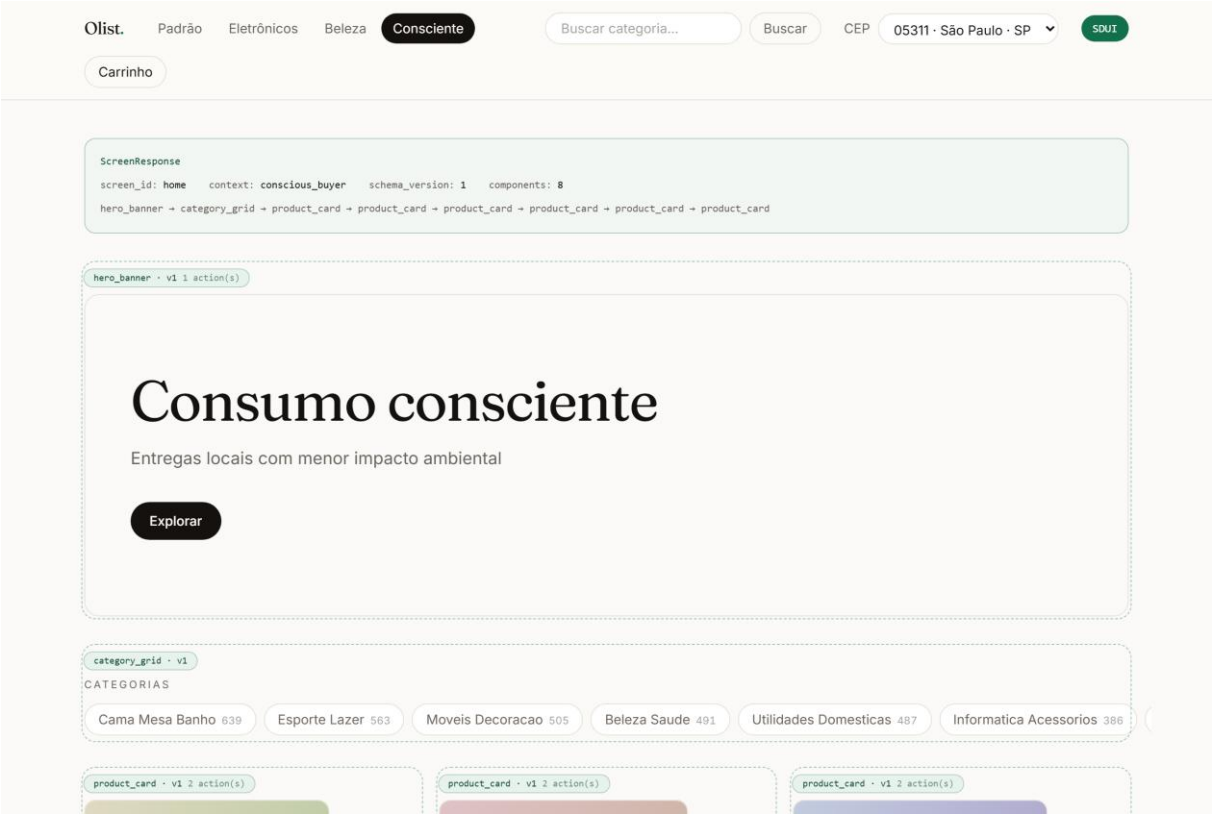
4.1.1 Requisitos funcionais

Os requisitos funcionais foram definidos a partir dos objetivos específicos do trabalho e das situações de uso previstas para o marketplace. O sistema deveria ser

capaz de adaptar a apresentação dos produtos ao contexto do comprador, levar em consideração a localização dos vendedores e apresentar uma jornada de compra simulada sem transformar o protótipo em uma plataforma de comércio eletrônico completa.

O primeiro requisito refere-se à composição da tela inicial. A rota da página inicial recebe o prefixo de CEP do comprador e devolve uma tela completa, representada pelo modelo "ScreenResponse", como pode ser visto na Figura 5.

Figura 5 – Página inicial no modo de inspeção SDUI



Fonte: Elaborada pelos autores (2026).

A tela é composta por um banner principal, uma grade de categorias e cartões de produtos. O contexto pode ser selecionado por perfis pré-definidos (como especialista em eletrônicos, beleza ou consumidor consciente) ou derivado automaticamente a partir do CEP informado, priorizando a categoria com maior lift[1]

[1] Métrica estatística que indica a sobre-representação de uma categoria em uma região. Representa a razão entre a proporção de vendas da categoria na UF analisada e a proporção de vendas dessa mesma categoria na média nacional. Valores superiores a 1 indicam concentração de consumo local acima do padrão do país.

regional na unidade federativa correspondente, desde que respeitados os limiares mínimos de volume da base.

Na Figura 5, pode-se observar que o quadro superior mostra a resposta enviada pelo servidor: o identificador da tela, o contexto aplicado e a sequência de blocos. Cada bloco da página aparece contornado e identificado pelo seu tipo.

Também foram previstos mecanismos de busca e filtragem. Como o conjunto de dados da Olist não apresenta um nome textual para cada produto, a busca é realizada sobre a categoria do produto. O termo informado pelo usuário passa por normalização de acentuação, caixa e separação de palavras. O filtro por categoria, por sua vez, utiliza o identificador da categoria presente no conjunto de dados. Dessa maneira, a interface não apresenta uma funcionalidade de busca por nome de produto que a base original não poderia sustentar.

O segundo requisito está relacionado à localização da oferta. Para cada produto considerado na composição da vitrine, calcula-se a distância entre o CEP do comprador e o CEP do vendedor por meio da fórmula de Haversine aplicada aos centroides geográficos. Ofertas com distância inferior a 100 km podem receber o selo de logística verde. O selo é acompanhado de uma indicação de distância e, quando há massa ou volume disponível, de uma estimativa de emissão. Na ausência de dados suficientes, o sistema omite o selo ou a informação correspondente, evitando apresentar uma recomendação ambiental sem fundamento numérico. O comprador também pode ordenar a vitrine por menor distância, menor emissão estimada ou menor preço; a ordenação é feita pelo servidor, que devolve uma nova árvore de componentes.

O terceiro requisito corresponde ao cálculo da pegada de carbono. O modelo de emissão evoluiu em relação ao fator único descrito no Capítulo 3, porque aplicar o fator de caminhão também à última milha, feita por veículos menores, subestimava a emissão das entregas locais. A emissão é estimada por uma abordagem baseada em atividade, compatível com o enquadramento da ISO 14083 e com os fatores do GLEC Framework. A distância geodésica é convertida em uma distância rodoviária aproximada por meio do fator de circuidade 1,345, com base em Lee e Chae (2023). O percurso é representado por dois trechos: uma etapa de transferência realizada por caminhão pesado e uma etapa final de até 15 km realizada por van, conforme a referência do World Economic Forum (2024). A massa utilizada no cálculo é a massa

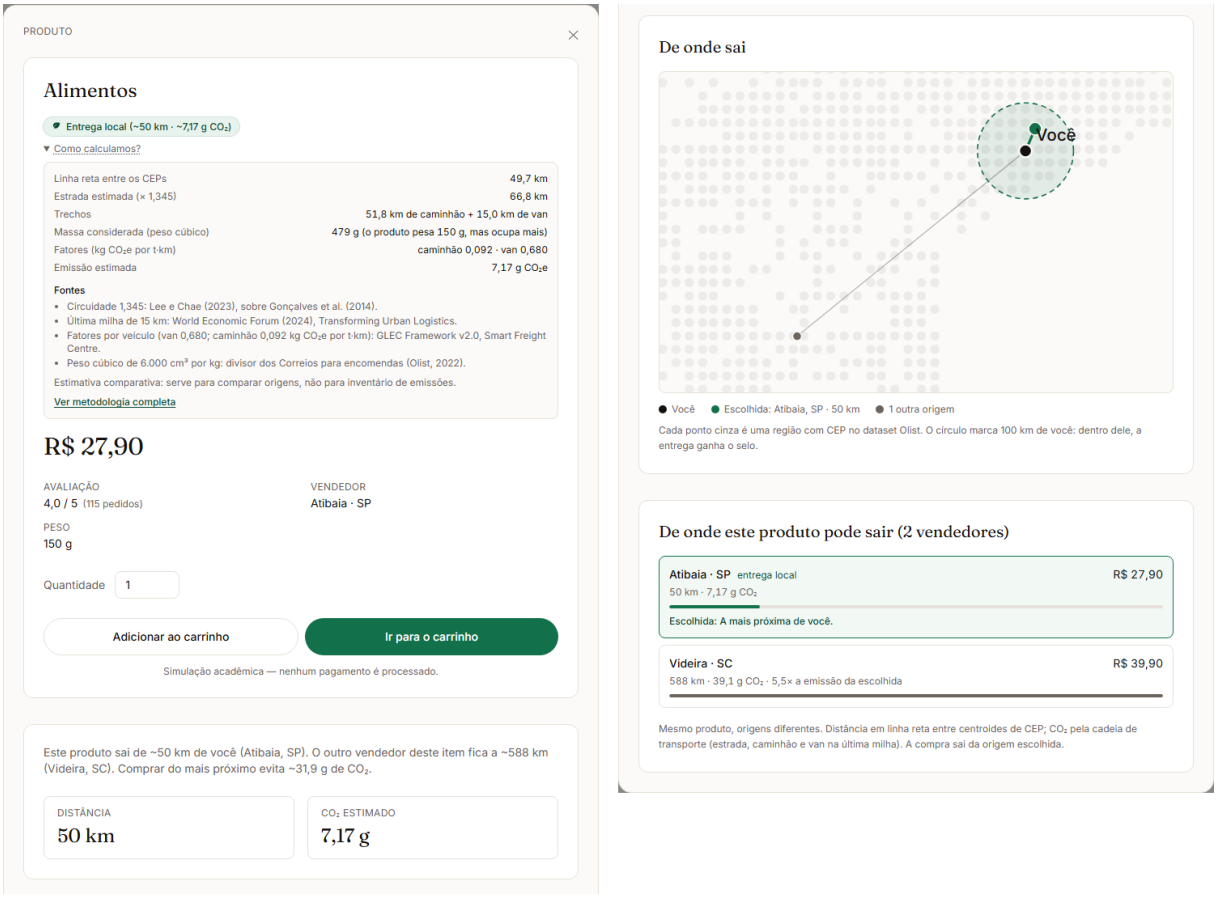
cobrável, isto é, o maior valor entre o peso real e o peso cubado. Para a cubagem, adotou-se a relação de 6.000 cm³ para cada quilograma (OLIST, 2022). Os fatores por veículo são os do GLEC Framework (SMART FREIGHT CENTRE, 2019). Para que a estimativa não seja uma caixa-preta, o selo oferece a opção "Como calculamos?", que abre a cadeia do cálculo da oferta exibida: distância em linha reta, estrada estimada, trechos de caminhão e de van, massa considerada, fatores e fontes (Figura 6).

O sistema também deveria manter uma regra única para a escolha da oferta. Um mesmo produto pode aparecer associado a mais de um vendedor e, por isso, a vitrine, a tela de detalhe e o checkout precisam apresentar uma escolha coerente. A regra prioriza a menor distância. Consideram-se praticamente equivalentes as ofertas cuja distância esteja dentro de uma tolerância de 1% da menor distância, com piso de 1 km. Nesse conjunto, o preço é utilizado como critério de desempate. O desempate nunca troca uma oferta com selo de logística verde por outra sem selo. Assim, pequenas diferenças que não são significativas para a aproximação geográfica não impedem a escolha de uma oferta mais barata.

Outro requisito foi a criação de uma tela de detalhe de produto orientada pelo servidor. A rota de catálogo dos produtos devolve uma nova ScreenResponse, composta principalmente pelos blocos de detalhe do produto e de impacto logístico. As avaliações são obtidas da agregação de dados reais de avaliações da Olist, enquanto a cidade e a UF do vendedor provêm da tabela de vendedores. Não foram incluídas descrições textuais ou etiquetas de sustentabilidade, pois esses dados não estão disponíveis no conjunto original.

Quando o produto é vendido por mais de um vendedor, a tela inclui um comparador com todas as origens, cada uma com preço, distância, emissão estimada e quantas vezes ela emite em relação à escolhida, e um mapa que mostra a origem escolhida, as alternativas e o raio de 100 km do selo (Figura 6). O comparador concretiza o que a metodologia previa na Seção 3.2: apresentar a redução relativa da opção de menor emissão em comparação com outra alternativa.

Figura 6 – Tela de detalhe do produto: selo com o cálculo aberto, mapa de origem e comparador de ofertas



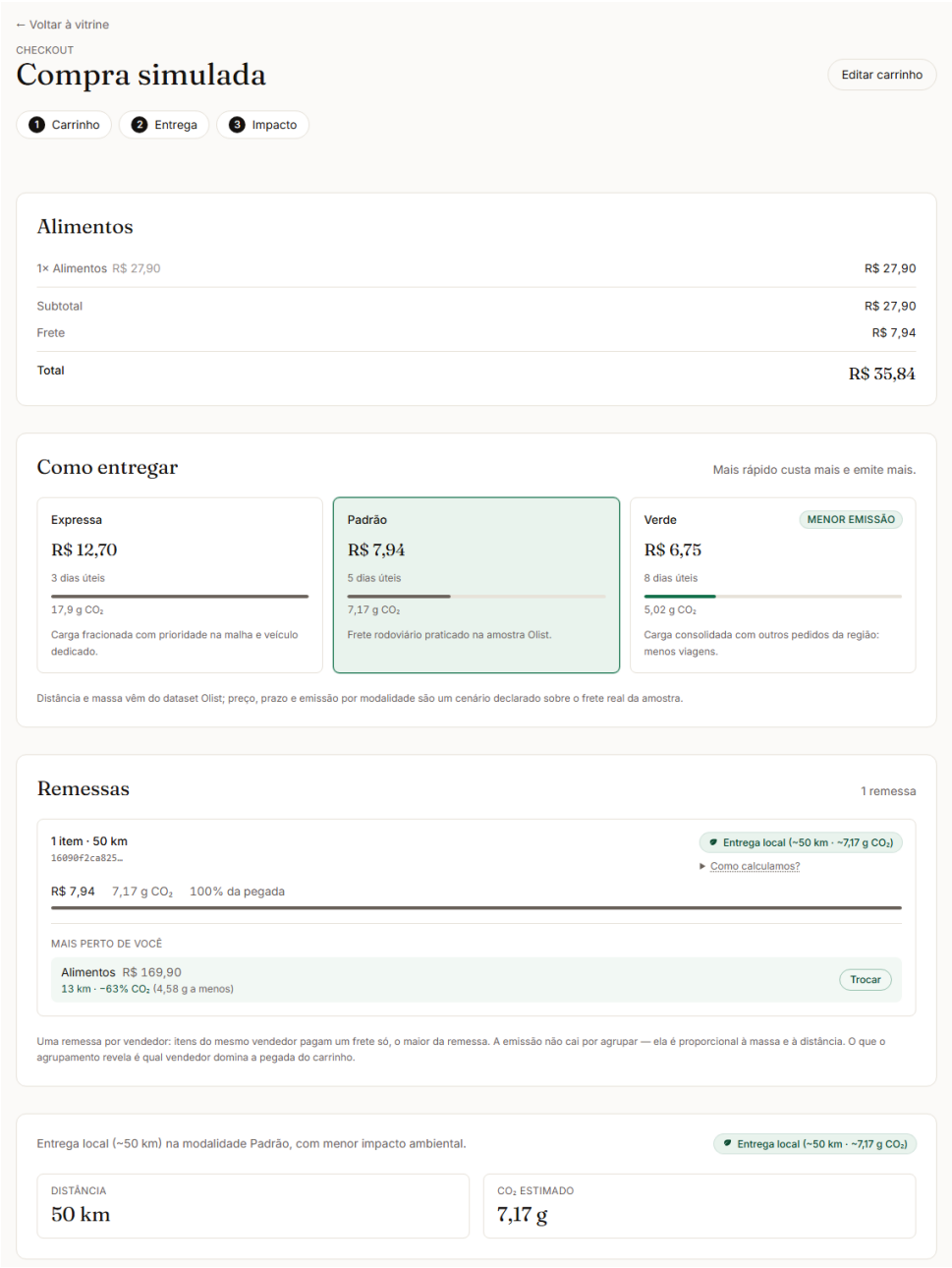
Fonte: Elaborada pelos autores (2026).

Para representar a etapa de compra, foi implementado um checkout simulado. A rota de simulação de checkout recebe uma lista de itens, suas quantidades e, opcionalmente, a modalidade de entrega. A resposta apresenta o resumo do carrinho, as opções de entrega, a divisão das remessas por vendedor e o impacto logístico estimado. O fluxo não realiza pagamento, não baixa estoque e não persiste um pedido.

O checkout também sugere trocas. Para a remessa de maior emissão, o sistema procura outro produto da mesma categoria vendido mais perto do comprador e informa quanto CO_2 a troca evitaria. Esse mecanismo é diferente da escolha de oferta: a escolha de oferta mantém o produto e muda o vendedor, enquanto a sugestão de troca muda o produto. Por isso, a troca é apresentada com preço e distância próprios, e a decisão fica com o usuário. As trocas aceitas somam-se em um indicador da sessão, "Seu impacto", que mostra o CO_2 evitado pelas escolhas do

próprio usuário. A Figura 7 mostra o checkout com as três modalidades de entrega, a remessa e uma sugestão de troca.

Figura 7 – Checkout simulado: resumo, modalidades de entrega, remessas e sugestão de troca



Fonte: Elaborada pelos autores (2026).

Modalidades de entrega. A modalidade padrão usa o frete observado na base e prazos medidos em entregas reais, agrupadas por faixa de distância. As modalidades expressa e verde são cenários declarados sobre essa linha de base, pois a Olist não informa transportadora, modal nem emissão por modalidade. A própria tela

avisa o usuário sobre essa distinção, e a página de metodologia do sistema apresenta os fatores com a justificativa de cada um (Tabela 3).

Tabela 3 – Fatores das modalidades de entrega em relação à linha de base

Modalidade	Preço	Prazo	CO_2	Justificativa
Padrão	× 1,0	× 1,0	× 1,0	Linha de base medida: frete da amostra e prazo mediano de 95.921 entregas reais, por faixa de distância
Expressa	× 1,6	× 0,5	× 2,5	Cenário sem fonte quantitativa: o veículo parte antes de completar a carga; o fator de CO_2 equivale a viajar com 40% da ocupação da entrega padrão
Verde	× 0,85	× 1,5	× 0,7	Consolidar pedidos reduz em 30% o número de viagens (WORLD ECONOMIC FORUM, 2024). Esperar a consolidação alonga o prazo, e um desconto compensa o comprador (WORLD ECONOMIC FORUM, 2024)

Fonte: Elaborada pelos autores (2026).

A literatura sustenta a direção dos fatores, não os valores: entregas mais rápidas reduzem a consolidação e aumentam as viagens, e entregas consolidadas fazem o contrário. Por isso, os números das modalidades expressa e verde são apresentados como cenário de comparação e não entram nos resultados de emissão do Capítulo 5, que usam apenas a linha de base.

Por fim, as ações da interface são descritas pelo servidor. Navegação, chamadas à API e abertura de modais são representadas como variantes do contrato de ações. Na versão implementada, o fluxo principal utiliza navegação e chamadas à API. A ação de abertura de modal permanece prevista no contrato, embora o detalhe de produto tenha passado a ser carregado como uma nova tela orientada ao servidor.

4.1.2 Requisitos não funcionais, métricas e critérios de aceitação

Além das funcionalidades diretamente visíveis ao usuário, o desenvolvimento considerou requisitos de desempenho, integridade do contrato, tolerância a dados incompletos e reprodutibilidade.

O desempenho foi estabelecido como uma meta experimental de tempo até o primeiro byte inferior a 200 ms nos cenários e percentis definidos na metodologia. Essa meta não foi tratada como uma garantia universal, pois o comportamento

depende do nível de concorrência e da condição do cache. A ferramenta de carga mede o tempo até a resposta completa, que é um limite superior do tempo até o primeiro byte; o critério adotado é, portanto, conservador. A avaliação deveria comparar três estados da camada de respostas em Redis, com cache desabilitado, frio e aquecido, além de considerar o tempo de reflexão entre as requisições. O acesso ao banco de dados da aplicação foi implementado de forma assíncrona, utilizando SQLAlchemy com asyncpg.

A integridade do contrato foi tratada com modelos Pydantic v2 e uniões discriminadas pelo campo type. No frontend, o contrato foi espelhado em TypeScript, e o registro de componentes React mantém a correspondência entre os tipos recebidos e os componentes renderizados. Essa correspondência é verificada por testes, pois uma divergência entre servidor e cliente poderia produzir uma tela incompleta mesmo quando cada parte estivesse funcionando isoladamente.

Também foi estabelecido que a interface deveria degradar de forma segura. Componentes desconhecidos não podem interromper a renderização da tela, e um envelope incompleto deve ser normalizado quando possível. Da mesma forma, a ausência de coordenadas, massa, avaliação ou alternativa válida deve resultar na omissão do campo correspondente. O sistema não deve substituir uma ausência de informação por um valor inventado.

Dois requisitos acompanharam a interface. O primeiro é a transparência das estimativas: toda emissão exibida pode ser decomposta pelo usuário, e uma página de metodologia, também enviada pelo servidor, apresenta os parâmetros e as fontes de cada fator. O segundo é a acessibilidade: a jornada completa, da entrada do CEP ao checkout, deve poder ser percorrida apenas com o teclado.

A reprodutibilidade foi considerada desde o tratamento dos dados até os ensaios de desempenho. A amostragem operacional utiliza semente fixa, os ensaios de carga registram seus parâmetros e as análises de concentração regional, emissões evitadas e comparação entre Haversine e OSRM são executadas por scripts específicos. As respostas obtidas do serviço público de roteamento foram preservadas em arquivo para que as métricas publicadas possam ser recalculadas sem depender de uma nova consulta à rede.

Os critérios de aceitação foram organizados em quatro grupos. O primeiro reúne os testes de distância, massa e emissão com entradas controladas. O segundo

verifica a ordenação das ofertas, das remessas e das alternativas. O terceiro confere a geração e a interpretação do contrato SDUI. O quarto avalia a jornada completa e o comportamento sob carga. Os resultados da aplicação desses critérios são apresentados no Capítulo 5.

4.1.3 Delimitações do escopo

O sistema foi desenvolvido como um protótipo de demonstração e avaliação, e não como uma plataforma comercial pronta para produção. A API utiliza uma amostra derivada do conjunto de dados da Olist, carregada em um PostgreSQL local após os filtros de qualidade. As análises de concentração logística e de CO_2 evitado, por sua vez, são executadas sobre o conjunto completo. Essa separação é necessária porque o volume utilizado para demonstrar a aplicação não é o mesmo volume utilizado para estudar o potencial logístico do conjunto de dados.

Ficaram fora do escopo a autenticação, os perfis de usuário, a persistência de pedidos, o pagamento, o painel do vendedor e as consultas espaciais com PostGIS. O módulo destinado a encaminhar diferentes versões de componentes também permanece como uma extensão futura. O campo version já existe no contrato, mas todos os componentes entregues utilizam a versão 1.

A distância utilizada pela aplicação continua sendo calculada entre centroides de prefixos de CEP. A comparação realizada com o OSRM mostrou que a estimativa corrigida pelo fator de circuidade apresenta erro menor do que a distância Haversine crua e preserva, em grande parte, a ordenação das ofertas. Entretanto, o OSRM representa a rota mais rápida de carro e não necessariamente a rota da transportadora. Além disso, os pontos de origem e destino são centroides, não endereços individuais. Por isso, as distâncias exibidas devem ser interpretadas como estimativas comparativas.

Também foram delimitadas as premissas das modalidades de entrega, em que agrupar itens de um mesmo vendedor reduz o frete simulado, mas não altera a pegada calculada (a emissão), que é estritamente linear em relação à massa transportada. A recomendação sustentável por proximidade restringe-se aos produtos que possuem ofertas concorrentes ativas.

4.2 PROJETO

Com os requisitos definidos, a etapa de projeto organizou as responsabilidades necessárias para transformar as regras de negócio em uma solução integrada. Foram estabelecidas a divisão do backend em fatias verticais, a estrutura relacional dos dados e a forma de comunicação entre o servidor e a interface. Essa etapa foi decisiva para manter próximas as decisões de personalização, os cálculos de logística e a composição visual, sem transferir ao cliente as regras que deveriam permanecer centralizadas no servidor.

4.2.1 Organização em Vertical Slices

Adotou-se a organização em fatias verticais para manter próximas as regras de negócio, a composição das telas e o acesso aos dados de cada funcionalidade. Essa decisão foi importante porque o sistema não se limita a consultar produtos: ele precisa combinar contexto, localização, emissão e apresentação em uma mesma resposta.

São cinco fatias:

a) home_contextual: composição da página inicial, contexto regional, busca e filtros;

b) green_logistics: centroides, distância, emissão, selo, escolha de oferta, modalidades de entrega e as telas de metodologia e de resultados da pesquisa;

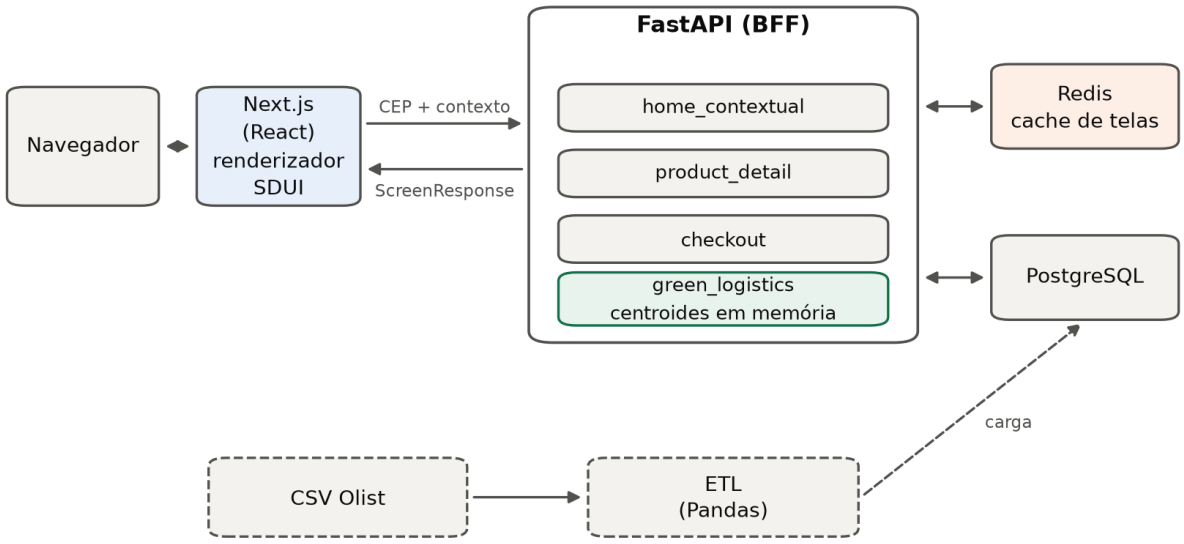
c) product_detail: tela de detalhe do produto;

d) checkout: simulação do carrinho, divisão em remessas e sugestões de troca;

e) orchestrator: mutações do contrato no modo de apresentação, usadas para demonstrar o desacoplamento ao vivo; o roteamento entre versões de componentes permanece como extensão futura.

O backend atua como uma camada dedicada à interface (BFF). Ele consulta os dados, aplica as regras e devolve a árvore de componentes pronta para o cliente. Uma mudança na ordenação dos produtos, por exemplo, acontece só no servidor; o cliente não precisa conhecer o cálculo de distância ou de emissão. A Figura 8 mostra a arquitetura implementada.

Figura 8 – Arquitetura implementada



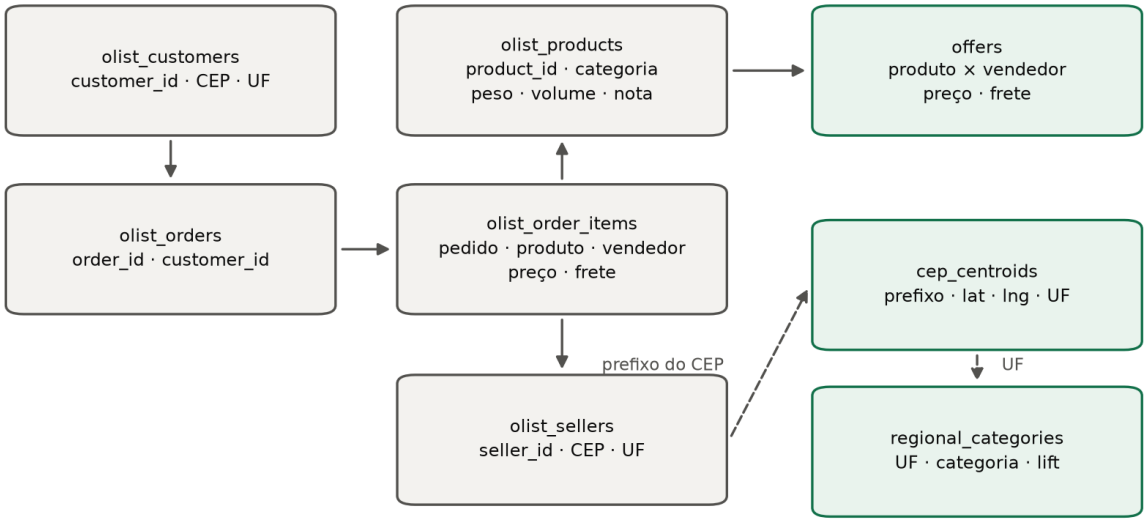
Fonte: Elaborada pelos autores (2026).

O navegador conversa apenas com o Next.js, que repassa o CEP e o contexto ao backend FastAPI e recebe de volta a ScreenResponse. O Redis guarda as telas já montadas, e o PostgreSQL guarda os dados carregados por um processo de ETL independente da API.

4.2.2 Modelo de dados relacional (PostgreSQL)

O PostgreSQL foi utilizado para persistir as entidades necessárias à aplicação e as tabelas derivadas durante o processo de preparação dos dados. As tabelas originais representam clientes, pedidos, itens de pedido, produtos e vendedores. As tabelas originadas do processo de preparação dos dados são cep_centroids, offers e regional_categories. Sobre essa base foram criadas estruturas que tornam possíveis as consultas e decisões específicas do sistema.

Figura 9 – Modelo de dados simplificado



Fonte: Elaborada pelos autores (2026).

As três tabelas derivadas aparecem em verde na Figura 9:

a) cep_centroids: representa cada prefixo de CEP pela mediana das coordenadas da tabela de geolocalização e pela UF mais frequente. Por ser estática durante a execução, a tabela é carregada uma única vez, na primeira requisição que precisa dela, e mantida em memória. Um mecanismo de sincronização impede cargas simultâneas.

b) offers: reúne as combinações de produto e vendedor, com preço e frete. Com ela, a vitrine deixa de depender da ordem em que os pedidos foram sorteados, e produtos com mais de uma oferta são comparados pela mesma regra no detalhe e no checkout.

c) regional_categories: guarda, para cada UF que atende aos limiares mínimos de volume, a categoria de maior lift, isto é, a razão entre a participação da categoria nas compras da UF e a participação dela no Brasil. O cálculo usa o conjunto completo, mas só concorrem categorias presentes no catálogo carregado.

O processo de carga é independente do tempo de execução da API. O script de ETL utiliza Pandas para ler e transformar os arquivos CSV e psycopg para gravar o resultado no PostgreSQL. A API, por outro lado, utiliza acesso assíncrono ao banco. Essa separação evita que as dependências e as operações de processamento em lote sejam incorporadas ao caminho de atendimento das telas.

4.2.3 Contrato SDUI (esquema e tipos de componentes)

O contrato de comunicação foi definido para que cada bloco da interface pudesse ser descrito pelo servidor sem que o cliente precisasse conhecer as regras que levaram à sua composição. Cada bloco possui o envelope "{type, version, props, actions}". O campo "type" identifica o componente, "version" permite evolução futura do contrato, "props" contém os dados de apresentação e "actions" descreve os comportamentos que o cliente deve executar.

A resposta completa é representada por "ScreenResponse", que reúne a versão do esquema, o identificador da tela, o contexto aplicado e a lista de componentes. A validação é feita com uniões discriminadas em Pydantic, o que permite distinguir as variantes pelo campo type e manter explícitos os campos de cada bloco. A Tabela 2 lista os doze componentes implementados.

Tabela 2 – Componentes do contrato SDUI

Componente (type)	**Tela**	**Função**
hero_banner	Página inicial	Destaque principal e motivo do contexto
category_grid	Página inicial	Categorias, cada uma com seu destino
product_card	Página inicial	Produto, preço e selo verde
product_detail	Detalhe	Dados do produto e inclusão no carrinho
impact_banner	Detalhe e checkout	Distância e CO_2 estimado
offer_comparison	Detalhe	Ofertas do mesmo produto por origem, com preço, distância e CO_2
origin_map	Detalhe	Mapa com a origem da oferta escolhida e das alternativas
checkout_summary	Checkout	Itens, frete e total
delivery_options	Checkout	Modalidades com preço, prazo e emissão
shipment_breakdown	Checkout	Remessas por vendedor e sugestões de troca
content_section	Metodologia e pesquisa	Texto explicativo enviado pelo servidor
data_table	Metodologia e pesquisa	Tabelas de parâmetros e resultados

Fonte: Elaborada pelos autores a partir do contrato implementado (2026).

Os quatro componentes de comparação, mapa, texto e tabela foram incluídos depois da primeira versão da jornada, sem alterar os anteriores. Os dois últimos compõem as telas de metodologia e de resultados da pesquisa, que expõem ao usuário as fontes e os fatores do cálculo e também são montadas pelo servidor.

O mapa e o detalhamento do cálculo, que o Capítulo 2 associa a micro-frontends, foram implementados como blocos do próprio contrato. O registro de componentes já isola cada bloco no cliente, e um micro-frontend acrescentaria um segundo ciclo de publicação sem benefício para um protótipo com um único cliente.

As ações disponíveis no contrato são navegação, chamada à API e abertura de modal. A ação de chamada à API pode solicitar uma nova tela por meio de uma requisição GET ou enviar os dados do carrinho para a simulação de checkout por meio de uma requisição POST. Em dois casos, as ações aparecem em níveis internos do bloco. Cada categoria possui seu próprio destino, e cada alternativa de remessa pode representar uma substituição diferente. Essa organização mantém explícita a relação entre o elemento visual e o comportamento correspondente.

No frontend, o registro de componentes associa cada type recebido ao componente React responsável por sua renderização. O contrato também possui um espelho em TypeScript. O cliente foi projetado para ignorar de forma segura um tipo que ainda não possua renderizador, sem transformar esse caso em uma falha da tela inteira. O modo de inspeção SDUI foi incluído para tornar visível o envelope recebido, seus tipos, suas ações e seus dados, o que facilita a verificação do desacoplamento proposto.

4.3 IMPLEMENTAÇÃO

A implementação correspondeu à materialização das decisões apresentadas nas etapas anteriores. Nela, os dados foram preparados para a execução da aplicação, as regras de distância, emissão e escolha de ofertas foram integradas às rotas do backend e as respostas foram transformadas em telas interpretadas pelo frontend. Também foram configurados os mecanismos de cache e o ambiente de execução, de modo que o mesmo artefato pudesse ser utilizado tanto na demonstração da jornada quanto nos procedimentos de avaliação descritos no capítulo seguinte.

4.3.1 Tratamento e carga do conjunto de dados Olist

O processo de preparação começa com a leitura dos arquivos CSV do conjunto de dados público da Olist. A amostragem utilizada pela API é realizada com semente fixa, de modo que a mesma configuração possa ser reconstruída. Depois da seleção inicial dos itens, o processo completa as ofertas dos produtos selecionados, chegando a 60.636 itens. Essa etapa é necessária porque a seleção linha a linha de pedidos poderia apresentar um produto sem revelar que ele também possui ofertas de outros vendedores.

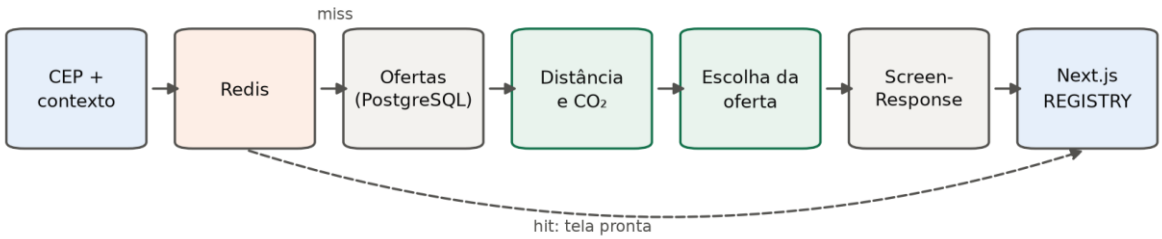
Em seguida, são construídos os centroides dos prefixos de CEP e é associada uma UF a cada prefixo quando essa informação está disponível. O ETL também deriva as ofertas, as categorias regionais, as avaliações agregadas e os campos necessários à composição do detalhe e do checkout. Registros sem peso utilizável ou sem geolocalização válida de cliente ou vendedor são retirados antes da persistência.

A escolha por calcular os centroides durante o ETL reduz o custo das consultas em tempo de execução. Da mesma forma, a derivação do contexto regional evita que a API tenha de calcular novamente a participação das categorias a cada acesso à página inicial. O resultado é um catálogo preparado para a demonstração, sem alterar os arquivos brutos do Olist.

4.3.2 BFF em FastAPI: motor de cálculo logístico e cache (Redis)

As rotas da aplicação são expostas pelo FastAPI e organizadas por fatias de responsabilidade. Quando a página inicial ou um checkout é solicitado, o backend valida o CEP, recupera os produtos e as ofertas disponíveis, calcula as distâncias e aplica a regra de escolha. Em seguida, o motor logístico calcula a emissão e os compositores transformam os dados em componentes SDUI, conforme observado na Figura 10.

Figura 10 – Fluxo de composição de uma tela



Fonte: Elaborada pelos autores (2026).

Quando a tela já está no Redis, a resposta volta pronta e o processamento avança diretamente sem passar pelas outras etapas, como pode ser visto na seta tracejada. Quando a tela não está pronta, a requisição percorre a consulta de ofertas, o cálculo de distância e emissão, a escolha da oferta e a montagem da ScreenResponse, que é então interpretada pelo registro de componentes no cliente.

O cálculo de distância consome diretamente a estrutura de centroides indexada na memória da aplicação. A primeira requisição carrega a tabela, e as seguintes consultam um dicionário, o que elimina idas repetidas ao banco durante a ordenação de vários produtos. O motor de emissão recebe a distância Haversine, aplica a circuidade de 1,345 e calcula a pegada a partir da massa cobrável, segregando o trajeto entre a última milha de até 15 km (fator de van) e o percurso restante (fator de caminhão pesado). O resultado é exibido em quilogramas ou gramas, conforme a grandeza, para que emissões pequenas não apareçam como zero.

O Redis guarda as respostas completas da página inicial por uma hora. Ele não substitui as otimizações do backend, porque mesmo sem Redis, os centroides ficam em memória e o PostgreSQL mantém seu cache de páginas. Por isso, nos ensaios, "cache desabilitado" significa apenas a camada de respostas em Redis. As telas de detalhe e de checkout não são guardadas em cache.

Para compor as alternativas no checkout, o sistema associa ao frete base os prazos de entrega estimados por faixas de distância, que são derivados a partir de 95.921 entregas reais da base, calculando multiplicadores objetivos para as modalidades simuladas por faixa de distância. Preço, prazo e emissão das modalidades expressa e verde derivam dessa linha de base por fatores declarados.

4.3.3 Composição das telas e renderização no cliente

Os compositores do servidor organizam os blocos de cada tela de acordo com o contexto e a funcionalidade solicitada. Na página inicial de consumidor consciente, por exemplo, os cartões são ordenados pela proximidade. No checkout, os itens são agrupados por vendedor, as remessas são ordenadas pela emissão estimada e as alternativas são selecionadas considerando distância, massa e potencial de redução de CO_2.

O frontend foi implementado com Next.js e utiliza React como biblioteca de renderização. O cliente recebe a ScreenResponse, consulta o registro de componentes e monta a tela sem repetir as regras de recomendação. A navegação para o detalhe e a chamada de simulação do checkout são executadas pelo interpretador de ações.

O carrinho é mantido no armazenamento local do navegador, pois não há persistência de pedidos neste escopo. Na leitura, seus dados são validados antes de serem utilizados. A ausência de um backend de pedidos não impede a avaliação da jornada, pois o objetivo desta etapa é verificar a composição server-driven e a simulação das consequências logísticas.

A mesma árvore de componentes é exibida em telas de celular, com o cabeçalho reduzido e os filtros recolhidos. A adaptação é feita no cliente, por folhas de estilo, sem que o servidor precise conhecer o dispositivo.

O contrato foi acompanhado por testes do renderizador, do interpretador de ações e dos casos de degradação. Também foram realizados testes de jornada com o navegador, incluindo alterações controladas na resposta SDUI. A análise dessas evidências pertence ao Capítulo 5, pois esta seção descreve apenas como o mecanismo foi construído.

4.3.4 Ambiente de execução (Docker Compose)

O ambiente foi organizado com Docker Compose para os serviços PostgreSQL e Redis. A API FastAPI é executada no hospedeiro e o frontend pode ser iniciado separadamente para a demonstração visual. Essa configuração foi mantida nas

medições porque o ambiente utilizado era Windows com Docker sobre WSL2. Dessa forma, a comunicação entre a API e o gerador de carga ocorreu por loopback, enquanto o banco e o cache permaneceram em contêineres no mesmo hospedeiro. Essa configuração difere da sub-rede de contêineres prevista no Capítulo 3; a diferença é declarada aqui porque mantém a comparação entre os estados do cache, mas acrescenta a disputa por processador entre API e gerador de carga.

Para executar a demonstração, primeiro são iniciados os serviços de persistência. Em seguida, o ETL carrega a amostra operacional e a API é iniciada. Quando o banco é recarregado enquanto a API permanece ativa, o snapshot de centroides e as respostas armazenadas no Redis precisam ser invalidados. Essa exigência decorre do caráter estático dos dados mantidos em memória durante a execução.

O projeto também possui verificações automatizadas para o backend e para o frontend. Os testes de carga e os testes de navegador dependem dos arquivos brutos e da pilha em execução, por isso são executados no ambiente de avaliação, enquanto os testes unitários, de contrato, de lint e de tipos podem ser executados no fluxo de integração contínua.

Com essa configuração, o sistema reúne o conjunto de dados preparado, o backend responsável pelas regras de negócio e o cliente responsável pela renderização. O capítulo seguinte apresenta as evidências obtidas a partir desse artefato, separando a qualidade dos dados, o comportamento funcional, as estimativas logísticas, o desempenho e a flexibilidade do contrato.

5 RESULTADOS

Este capítulo apresenta os resultados obtidos com a implementação descrita no capítulo anterior. A análise foi organizada para responder a uma pergunta central: em que medida o artefato construído consegue combinar personalização contextual, escolha de ofertas próximas, estimativa de impacto ambiental e flexibilidade de apresentação sem comprometer o funcionamento da jornada de compra?

Os resultados foram divididos em duas dimensões. A primeira é quantitativa e reúne evidências sobre a cobertura dos dados, o comportamento do motor de cálculo, a aproximação da distância rodoviária, o potencial estimado de redução de emissões, a personalização regional e o desempenho da aplicação sob carga. A segunda é qualitativa e examina aquilo que não pode ser resumido por uma única métrica, como a capacidade de alterar a árvore de componentes, a tolerância a blocos desconhecidos e a correspondência entre o contrato produzido pelo backend e o contrato interpretado pelo frontend.

Essa separação não significa que as duas dimensões sejam independentes. Os resultados quantitativos dependem do tratamento dos dados e das regras implementadas no backend, enquanto os resultados estruturais indicam se essas regras podem ser apresentadas e alteradas sem reescrever a interface. Por isso, cada seção apresenta primeiro o procedimento de avaliação, depois os valores observados e, por fim, uma interpretação relacionada aos objetivos do trabalho.

É importante distinguir três tipos de resultado apresentados ao longo do capítulo. Alguns valores são observações diretas do conjunto de dados, como a quantidade de pedidos e a distância dos casos avaliados. Outros são resultados de um modelo, como a emissão de CO_2 calculada a partir de distância e massa. Há ainda resultados contrafactuais, como a emissão que seria obtida caso o comprador tivesse escolhido uma oferta alternativa. Essa distinção impede que uma estimativa seja apresentada como medição operacional.

5.1 RESULTADOS QUANTITATIVOS

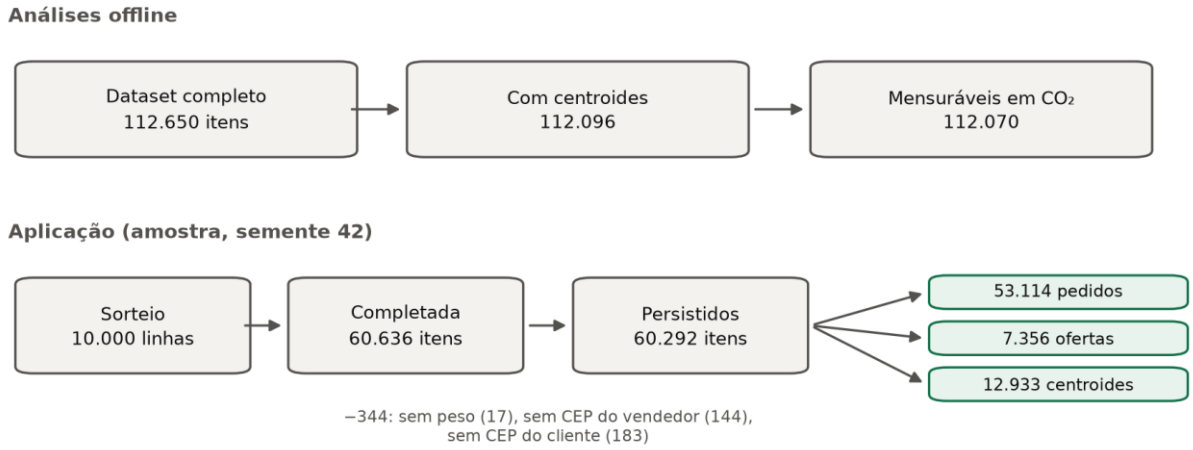
Os resultados quantitativos foram organizados de forma progressiva, partindo da qualidade da base utilizada e chegando ao comportamento da aplicação sob carga.

Essa sequência permite compreender primeiro quais dados sustentam as análises, depois verificar se as regras foram implementadas conforme o modelo definido e, por fim, observar os efeitos da solução em escala experimental. Ao longo da seção, os valores são acompanhados de sua base de cálculo e de suas limitações, para que não sejam confundidas medições diretas, estimativas do modelo e projeções contrafactuais.

5.1.1 Qualidade e cobertura dos dados

Os resultados usam dois universos de dados (Figura 11). As análises de logística verde partem do conjunto completo da Olist: 112.650 itens vendidos, dos quais 112.096 têm coordenadas para comprador e vendedor e 112.070 reúnem todos os campos do cálculo de emissão. A aplicação, por sua vez, usa uma amostra de 60.636 itens, obtida pelo sorteio de 10.000 linhas e completada com todas as ofertas dos produtos sorteados.

Figura 11 – Universos de dados das análises e da aplicação



Fonte: Elaborada pelos autores a partir dos resultados do ETL (2026).

Na amostra, 344 itens (0,57%) foram descartados por falta de peso ou de geolocalização, e 60.292 foram gravados, com 53.114 pedidos, 7.356 ofertas e 12.933 centroides de CEP. A perda é pequena, mas o critério importa: o sistema não atribui coordenadas padrão nem calcula emissão sem massa.

A complementação das ofertas foi decisiva. Sem ela, a amostra teria apenas 106 produtos com mais de um vendedor; com ela, passou a ter 580, antes dos

descartes. Uma avaliação de proximidade só tem sentido quando há alternativas: se a amostra escondesse as outras ofertas, o sistema pareceria escolher o vendedor mais próximo apenas por falta de opção. Como o sorteio é feito por linha de pedido, a amostra favorece os produtos mais vendidos e não deve ser usada para estimar proporções do catálogo inteiro.

Duas ressalvas acompanham todos os resultados. Os centroides representam um prefixo de CEP, não um endereço, o que introduz incerteza nas distâncias. E a base é histórica: a presença de um vendedor indica que a venda ocorreu entre 2016 e 2018, não que a oferta estaria disponível no momento da recomendação.

5.1.2 Verificação funcional do motor de cálculo

Antes das análises agregadas, as regras do motor foram verificadas com entradas controladas, como prevê a metodologia: distância de Haversine, conversão para distância rodoviária, massa cobrável, divisão entre transferência e última milha, fatores de emissão, escolha de oferta e ausência de recomendação quando faltam dados. Na versão final, a suíte do backend somava 229 testes aprovados e a do frontend, 42, além de 7 testes de jornada no navegador.

Um caso ilustra a cadeia de cálculo. Para 100 km em linha reta e 1 kg de massa, o motor estima 134,5 km de estrada: 15 km de van, a 0,680 kg de CO_2e por tonelada-quilômetro, e 119,5 km de caminhão pesado, a 0,092. O resultado é 0,021 kg de CO_2e. Os testes também confirmam que a emissão nunca diminui quando a distância ou a massa aumentam, e que o peso cubado de uma remessa é calculado sobre o conjunto dos itens. Somar o maior valor entre peso real e cubado item a item cobraria duas vezes o espaço ocupado e superestimaria a massa.

Esses testes demonstram consistência interna, não precisão externa. Validar os fatores de emissão exigiria dados de transportadoras, como veículo, rota, ocupação e consumo, que a Olist não fornece.

5.1.3 Acurácia da distância: Haversine e OSRM

A metodologia previu comparar a distância em linha reta com a distância de rota calculada pelo OSRM. Foram usados 100 pares de compras reais, 20 em cada faixa de distância, entre os mesmos centroides usados pela aplicação. A Tabela 4 resume a comparação.

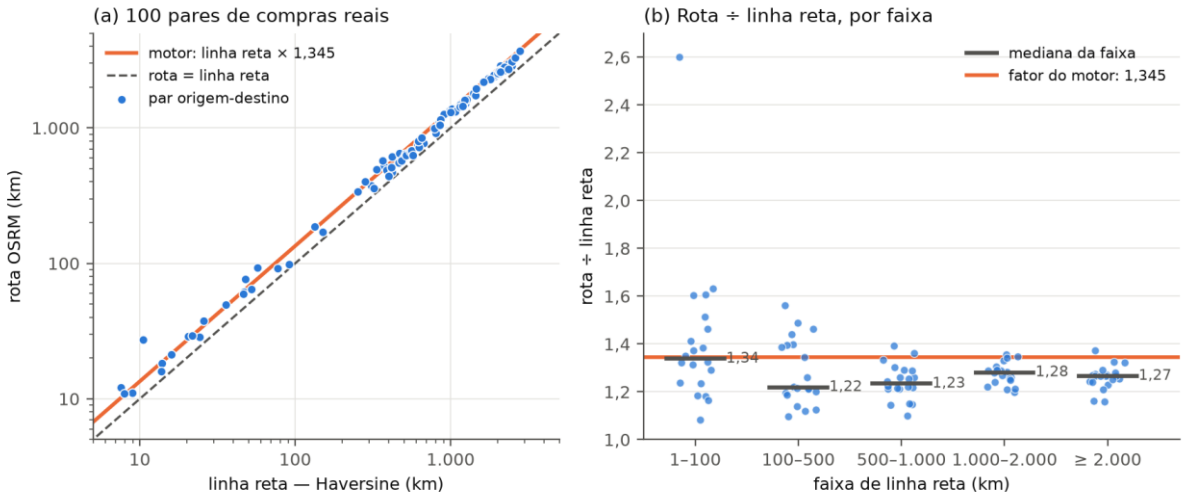
Tabela 4 – Erro da distância estimada em relação à rota do OSRM

Medida	Haversine crua	Haversine corrigida
MAE em relação ao OSRM	248,9 km	83,2 km
MAPE em relação ao OSRM	21,9%	8,7%
Viés mediano	Subestima em todos os pares	+6,2% após correção
Correlação de Spearman	0,997	0,997

Fonte: Elaborada pelos autores (2026).

A linha reta subestima a estrada em cerca de 22%. Com o fator de circuidade, o erro cai para 8,7%. A Figura 12 mostra por que o viés restante é positivo: até 100 km, a razão entre rota e linha reta (1,34) coincide com o fator adotado; acima disso, as estradas são mais retas (1,22 a 1,28), e o motor superestima a distância, e, portanto, a emissão, em 5% a 10%. O resultado é coerente com a fonte do fator, estimado para distâncias menores (LEE; CHAE, 2023).

Figura 12 – Distância em linha reta e distância de rota



Fonte: Elaborada pelos autores a partir do OSRM (2026).

Para o marketplace, porém, o que importa é a ordem dos vendedores. Em 37 decisões reais, envolvendo 102 trajetos entre comprador e vendedor, o vendedor mais próximo em linha reta também foi o mais próximo pela estrada em 32 (86,5%). Nas cinco divergências, a escolha pela linha reta acrescentou no máximo 6,4 km de

estrada (2,1% da rota), e em três delas a própria regra de escolha já tratava as ofertas como equivalentes. Com apenas 37 decisões, a incerteza desse percentual é grande (intervalo de confiança de 95% entre 72% e 94%).

O fator de circuidade é, portanto, uma aproximação empiricamente apoiada: reduz o erro e preserva a decisão de proximidade na maioria dos casos. As distâncias exibidas continuam sendo estimativas comparativas, pois o OSRM calcula a rota mais rápida de carro, não a rota da transportadora.

5.1.4 Potencial estimado de redução de emissões

Esta seção usa o conjunto completo da Olist para responder a duas perguntas: existe oferta próxima para os compradores, e quanto CO_2 seria evitado se o vendedor mais próximo fosse escolhido? A Tabela 5 reúne os resultados.

Tabela 5 – Proximidade e potencial de redução de emissões

Indicador	Resultado
Itens com comprador e vendedor na mesma UF	36,3%
Compras com vendedor a menos de 100 km	18,7%
Compradores com algum vendedor a menos de 100 km	94,7%
Itens cujo produto tinha mais de um vendedor	14.367 (12,8%)
Desses, comprador já escolheu o mais próximo	48,3%
Distância mediana desses itens: escolha real → mais próxima	437 → 330 km
CO_2 evitado nesses itens	254,2 kg CO_2e (9,8%)
CO_2 evitado sobre todas as compras mensuráveis	0,9%

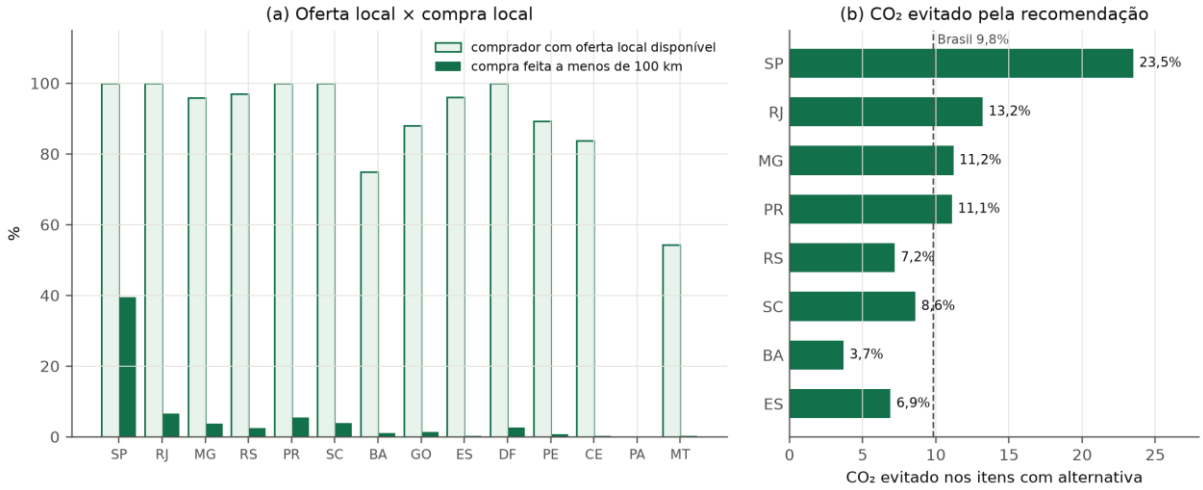
Fonte: Elaborada pelos autores a partir das análises de concentração regional e CO_2 evitado (2026).

Os três primeiros indicadores mostram uma lacuna. Quase todos os compradores (94,7%) tinham algum vendedor a menos de 100 km, mas só 18,7% das compras foram locais. Proximidade estadual também não é proximidade real: uma compra na mesma UF pode percorrer centenas de quilômetros.

A média nacional esconde diferenças regionais (Figura 13a). Em SP, RJ, PR, SC e DF, todos os compradores tinham oferta local; na BA, 74,9%; no MT, 54,3%; e no PA, nenhum. A distância mediana das compras vai de 163 km em SP a 2.391 km no PA. A oferta local, quando existe, também não garante a compra local: o RJ tinha

vendedor a menos de 100 km para todos os compradores e só 6,6% de compras locais, contra 39,5% em SP. A lacuna, portanto, é de escolha, e não apenas de oferta, o que justifica uma interface que torne a proximidade visível.

Figura 13 – Oferta local, compra local e CO_2 evitado por UF: (a) compradores com oferta local disponível e compras feitas a menos de 100 km; (b) CO_2 evitado nos itens com alternativa



Fonte: Elaborada pelos autores a partir das análises de concentração regional e CO_2 evitado (2026).

A recomendação, no entanto, só pode agir quando o mesmo produto é vendido por mais de um vendedor, o que ocorre em 12,8% dos itens. Nesses casos, metade dos compradores já havia escolhido o mais próximo. Se todos seguissem a recomendação, a distância mediana cairia de 437 para 330 km, e o modelo estima 254,2 kg de CO_2e evitados: 9,8% da emissão desses itens, ou 0,9% de todas as compras mensuráveis. O primeiro número descreve o potencial onde a regra pode agir; o segundo dimensiona o efeito sobre a base inteira.

O potencial também se distribui de forma desigual (Figura 13b). Nos itens com alternativa, a recomendação evitaria 23,5% da emissão em SP e 3,7% na BA, e SP responde sozinho por 107,2 dos 254,2 kg evitados (42%). A regra rende mais onde já há mais vendedores, e o efeito nacional de 9,8% é puxado por SP.

O custo para o comprador foi pequeno. Nos 7.430 itens em que a recomendação trocaria de vendedor, a oferta mais próxima era mais barata em 43,3% dos casos, de mesmo preço em 6,3% e mais cara em 50,4%, com diferença mediana de R$ 0,01. Como metade dos casos fica mais cara, a interface deve exibir distância, emissão e preço juntos, em vez de esconder a decisão sob um selo.

A aplicação não usa a oferta mais próxima pura, e sim uma regra com desempate: entre ofertas a até 1% da menor distância (com piso de 1 km), escolhe a mais barata, sem trocar uma oferta com selo por uma sem. Repetida com essa regra, a análise muda a escolha em 1.823 itens (12,7%). Nesses itens, a distância a mais é nula na mediana, porque são vendedores do mesmo prefixo de CEP, e o preço cai R$ 6,11 na mediana. O CO_2 evitado passa de 254,2 para 253,7 kg CO_2e e continua em 9,8%. Nos itens em que a regra trocaria o vendedor, a oferta recomendada fica mais cara em 39,3% dos casos, contra 50,4% sem o desempate. O desempate por preço reduz o custo da recomendação sem alterar o benefício ambiental.

Esses números são contrafactuais e devem ser lidos como potencial. O cálculo supõe que a oferta mais próxima estaria disponível e que só a origem mudaria, sem modelar estoque, consolidação de carga ou tipo de veículo. E, como o fator de circuidade superestima as distâncias longas (Seção 5.1.3), os valores absolutos de emissão podem estar até cerca de 10% acima; a ordem de grandeza e a conclusão não mudam.

Para verificar quanto esses resultados dependem dos parâmetros do modelo, a análise foi repetida variando cada um em 20% para baixo e para cima: circuidade, extensão da última milha, fator de emissão do caminhão, fator da van e divisor de cubagem. Também foram testados valores alternativos com fonte: a circuidade de 1,27 medida contra o OSRM (Seção 5.1.3), a última milha de 20 km (WORLD ECONOMIC FORUM, 2024) e a cubagem rodoviária de 300 kg/m³. A emissão absoluta varia de −23% a +67% entre os cenários, e o maior efeito vem da cubagem rodoviária. A parcela evitada, porém, fica entre 9,5% e 10,0% em todos eles, contra 9,8% na referência. Em nenhum cenário a recomendação emite mais que a escolha real, porque o mesmo produto, vindo de mais perto, emite menos para qualquer valor positivo dos parâmetros. Os números absolutos dependem das escolhas do modelo; a conclusão relativa, não.

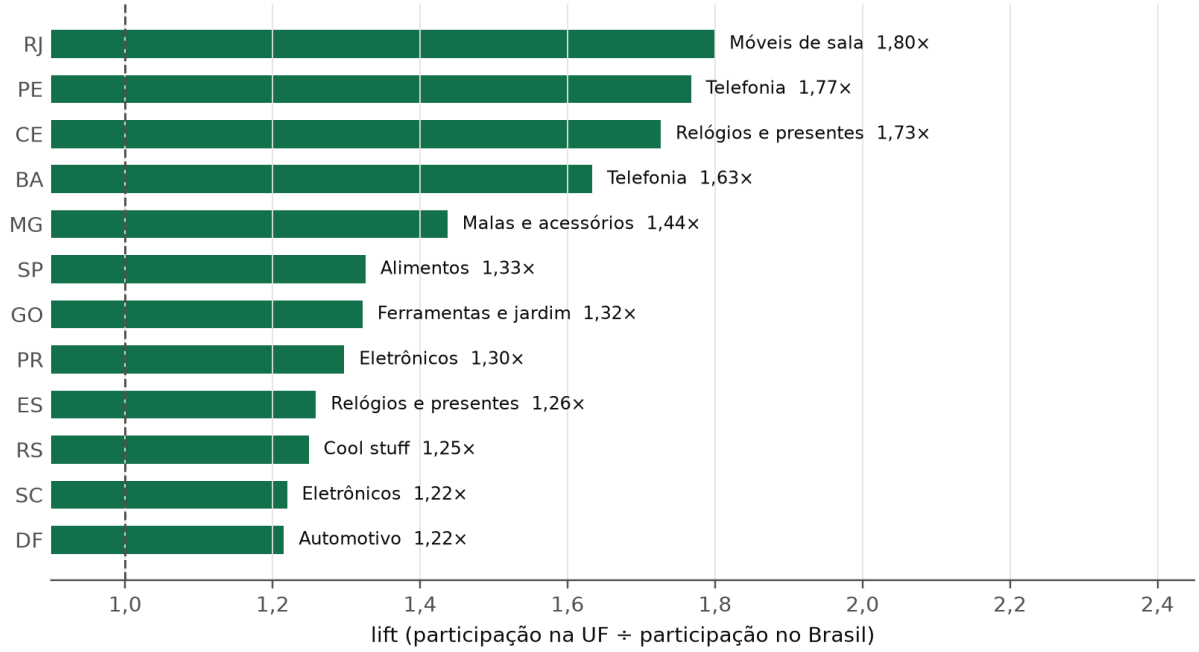
5.1.5 Personalização contextual regional

Sem perfil escolhido, a página inicial destaca a categoria que a UF do comprador compra acima da média nacional. A medida é o lift: a participação da categoria nas compras da UF dividida pela participação dela no Brasil. Para evitar

ruído, só são personalizadas UFs com pelo menos 1.000 itens vendidos, categorias com pelo menos 100 itens na UF e lift mínimo de 1,2.

Com esses limiares, 12 das 27 UFs receberam personalização (Figura 14), com lifts entre 1,22 e 1,80. Os sinais mais fortes estão em móveis de sala no Rio de Janeiro, telefonia em Pernambuco e na Bahia e relógios e presentes no Ceará.

Figura 14 – Categoria destacada por UF e intensidade do sinal regional



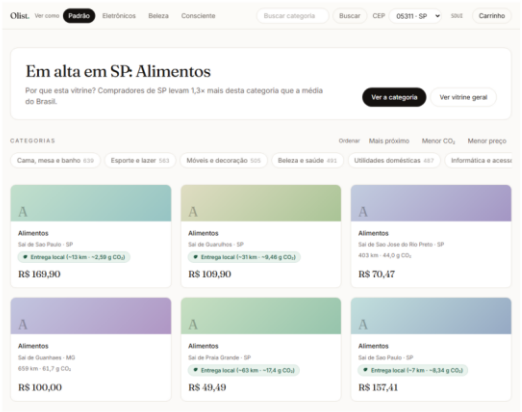
Fonte: Elaborada pelos autores a partir da tabela de categorias regionais (2026).

As 15 UFs restantes, incluindo todas as do Norte e boa parte do Nordeste, não têm volume suficiente e recebem a vitrine geral. A personalização acontece no nível da região, e não do indivíduo: sem login nem histórico de compras, o sistema não conhece o comprador, apenas o seu CEP. Os limiares protegem contra ruído, mas não constituem um teste de significância estatística. O custo de desempenho foi nulo: a UF e a tabela regional ficam em memória, e a mediana do tempo de resposta foi de 9,3 ms com a personalização e 9,5 ms sem ela.

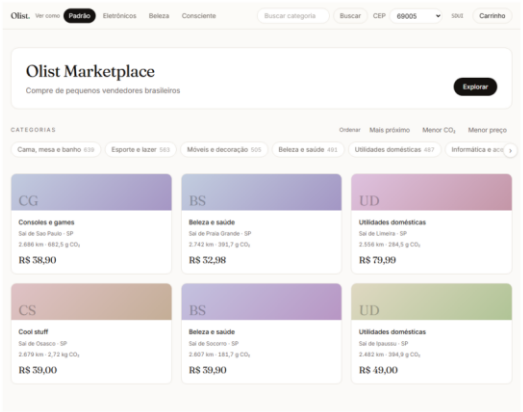
A Figura 15 mostra a diferença na tela. Com o CEP de São Paulo, a vitrine abre com a categoria em destaque e o motivo da escolha; com um CEP do Amazonas, sem volume suficiente, abre a vitrine geral.

Figura 15 – Página inicial com contexto regional (SP) e sem sinal regional (AM)

(a) CEP 05311 (SP): categoria em destaque



(b) CEP 69005 (AM): vitrine geral



Fonte: Elaborada pelos autores (2026).

5.1.6 Desempenho sob carga e efeito do cache

O desempenho foi medido sobre a jornada completa, e não sobre uma chamada isolada. A jornada inclui a página inicial em quatro variações de contexto, o detalhe do produto e o checkout. O protocolo seguiu a metodologia: três níveis de carga (50, 250 e 1.000 usuários virtuais), três estados do cache de respostas em Redis e três repetições, totalizando 27 ensaios válidos. Os ensaios rodaram em um computador com Windows 11, 8 núcleos lógicos e 7,9 GB de memória, com PostgreSQL 16 e Redis 7.4 em contêineres e a API (Python 3.13 e FastAPI) e o Locust 2.46 no hospedeiro; cada execução registrou o commit, as imagens, as versões e o volume de dados em um arquivo de metadados. A Tabela 6 apresenta as médias e os desvios padrão.

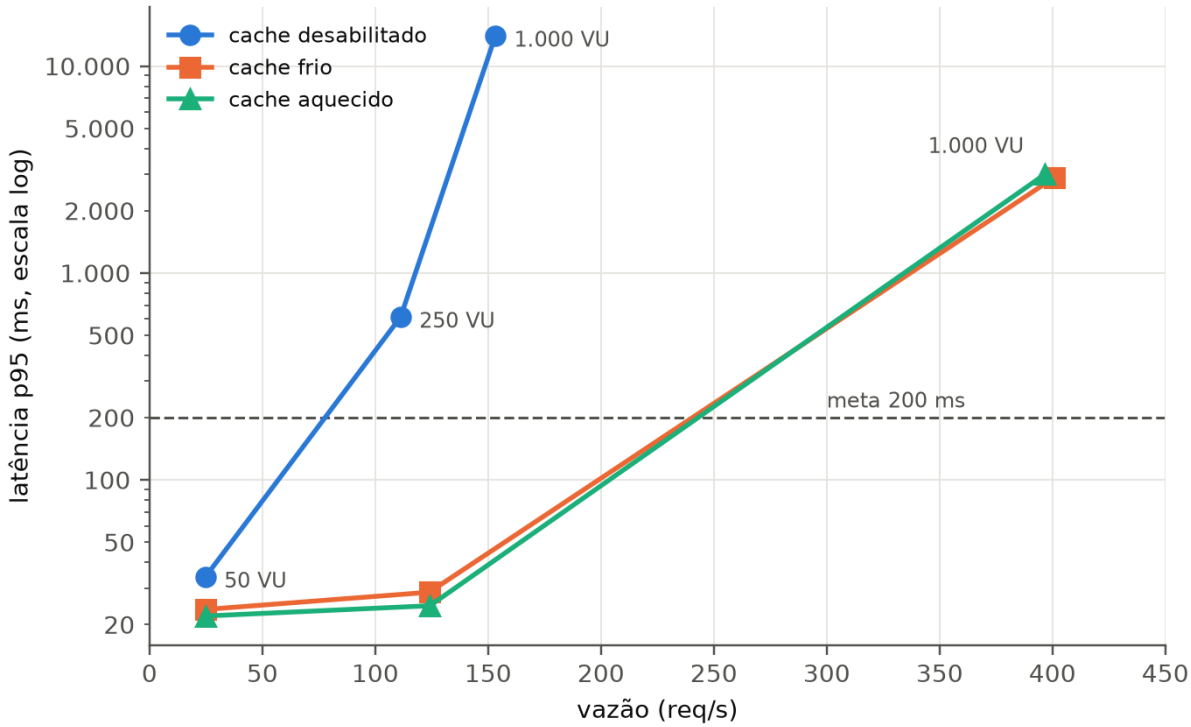
Tabela 6 – Desempenho da jornada por nível de carga e estado do cache

Usuários virtuais	Cache	p50 (ms)	p95 (ms)	p99 (ms)	Vazão (req/s)	Falhas
50	Desabilitado	12,7 ± 0,6	34 ± 6	46 ± 11	24,8 ± 0,1	0%
50	Frio	6,3 ± 0,6	24 ± 2	39 ± 8	24,9 ± 0,0	0%
50	Aquecido	6,7 ± 0,6	22 ± 2	36 ± 2	24,9 ± 0,1	0%
250	Desabilitado	173 ± 6	613 ± 29	1.030 ± 214	111 ± 0	0%
250	Frio	5,7 ± 0,6	29 ± 8	96 ± 83	124 ± 0	0%
250	Aquecido	5,7 ± 0,6	25 ± 6	51 ± 17	124 ± 0	0,02%
1.000	Desabilitado	4.100 ± 400	14.000 ± 1.732	22.000 ± 3.464	153 ± 13	0,21%
1.000	Frio	93 ± 15	2.900 ± 1.513	6.567 ± 3.066	401 ± 35	0,03%
1.000	Aquecido	95 ± 7	3.000 ± 1.058	7.167 ± 2.515	397 ± 27	0,07%

Fonte: Elaborada pelos autores a partir dos ensaios de carga (2026). Média ± desvio padrão de três repetições.

A Figura 16 mostra o mesmo resultado em forma de curva: o ponto em que a latência dispara indica a capacidade do sistema.

Figura 16 – Latência p95 em função da vazão, por estado do cache



Fonte: Elaborada pelos autores a partir dos ensaios de carga (2026).

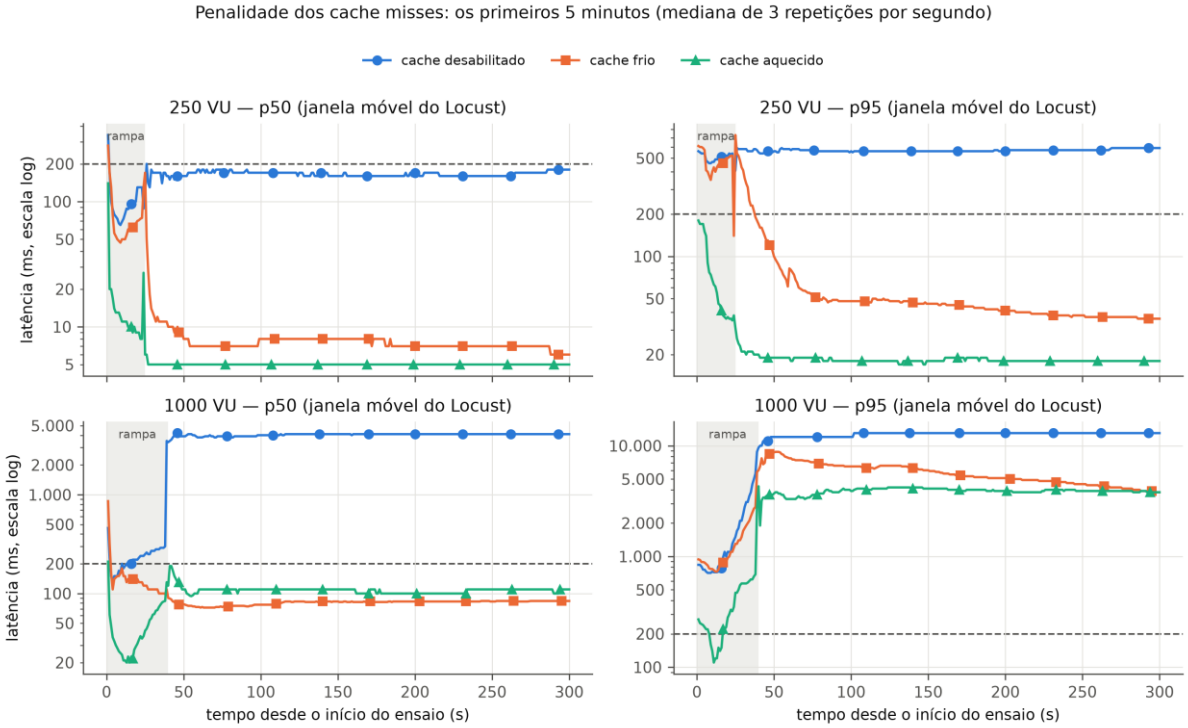
Com 50 usuários, a meta de 200 ms foi cumprida nos três estados. Nesse nível, o cache é uma otimização, não uma necessidade.

Com 250 usuários, o cache decide. Sem ele, o processador da API chega a 91% de uso, as requisições entram em fila e o p95 sobe para 613 ms. Com o cache, o mesmo equipamento atende toda a demanda com p95 entre 25 e 29 ms e um terço do processamento. A vazão parece pouco diferente (111 contra 124 req/s) porque os usuários simulados esperam a resposta antes de pedir a próxima tela: sem cache, eles passam mais tempo esperando e pedem menos.

Com 1.000 usuários, o sistema satura nos três estados, mas não da mesma forma. Sem cache, a mediana chega a 4,1 s. Com cache, a vazão é 2,6 vezes maior (cerca de 400 req/s) e a mediana fica abaixo de 100 ms. O p95 agregado, porém, continua em torno de 3 s, porque as telas de detalhe e de checkout não passam pelo cache e formam a fila. A rota cacheada isolada, a página inicial do consumidor consciente, ficou com p95 de 190 a 200 ms: no limite da meta, e não abaixo dela.

Os estados frio e aquecido são praticamente iguais nas médias. A diferença aparece apenas nos primeiros 1 a 3 minutos do ensaio frio, enquanto o Redis se preenche, e desaparece em seguida (Figura 17).

Figura 17 – Evolução temporal da latência nos primeiros 5 minutos, por estado do cache (mediana de três repetições)



Fonte: Elaborada pelos autores a partir dos ensaios de carga (2026).

Um ensaio complementar verificou se mais processos da API substituiriam o cache. Sem cache e com 1.000 usuários, quatro processos elevaram a vazão de 170 para 284 req/s (1,7 vez), mas ocuparam cerca de três núcleos. O cache alcançou cerca de 400 req/s com um único processo. Nesta máquina, portanto, mais processos não substituem o cache.

Três limites acompanham esses números. API, banco, cache e gerador de carga dividiram o mesmo computador, então os resultados comparam configurações entre si, mas não representam um serviço distribuído em produção. Com três repetições, a dispersão na carga máxima é alta (p95 com desvio de 1 a 1,7 s). E os ensaios foram executados antes das últimas mudanças de regra do sistema (motor de emissão em cadeia, contexto regional e regra única de oferta). Por isso, o cenário de 250 usuários foi repetido na versão final, nos três estados do cache e com três repetições. O custo de processamento por requisição ficou igual (8,5 contra 8,3 ms sem cache), e com cache o p95 ficou em 19 ms, abaixo dos 25 a 29 ms originais. Sem cache, o p95 variou entre 600 e 1.200 ms entre as repetições: com o processador acima de 90%, pequenas variações formam filas longas. A conclusão sobre o cache se mantém. Por fim, catorze ensaios da primeira execução foram descartados e

refeitos: treze porque o computador funcionou na bateria, com o processador em frequência reduzida, e um porque o próprio gerador de carga saturou.

5.2 RESULTADOS QUALITATIVO-ESTRUTURAIS

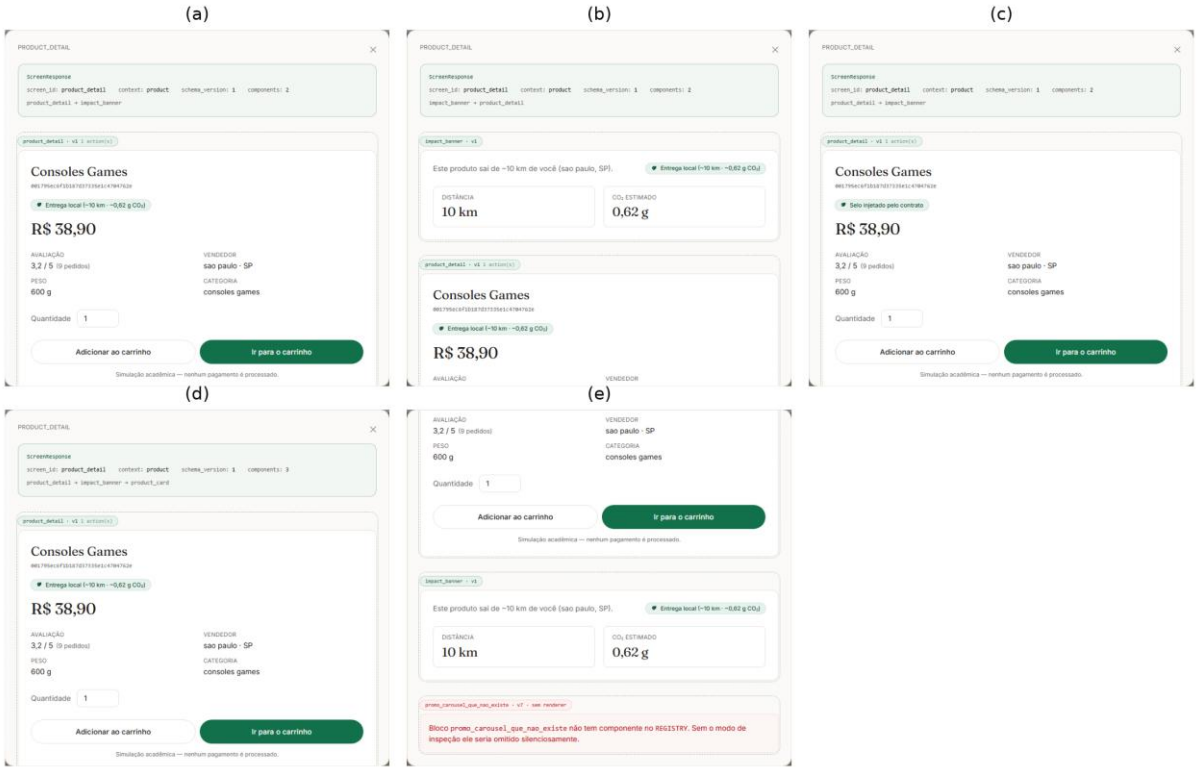
Os resultados qualitativos complementam as métricas anteriores ao examinar a forma como a solução se comporta diante de alterações no contrato e de situações que não podem ser reduzidas a um valor de latência ou emissão. Nesta etapa, a avaliação concentrou-se na capacidade de o frontend interpretar composições modificadas pelo servidor e de preservar a jornada quando recebe blocos desconhecidos ou dados insuficientes. O foco, portanto, recai sobre o desacoplamento e a robustez que sustentam a proposta de uma interface orientada pelo servidor.

5.2.1 Desacoplamento por mutação do contrato

Para verificar a flexibilidade da interface, um teste automatizado no navegador interceptou a resposta real do servidor para a tela de detalhe e aplicou cinco alterações ao JSON antes de entregá-lo ao cliente. As alterações foram reordenar os blocos, injetar um selo, incluir um bloco conhecido que a tela não tinha, incluir um tipo de bloco inexistente e remover o campo actions de um bloco. O critério de sucesso foi a tela refletir a alteração sem novo build e sem recarregar a página, o que foi comprovado por um marcador gravado no navegador antes da primeira mutação e conferido ao final.

As cinco mutações foram aprovadas (Figura 18). A reordenação mostrou que a ordem dos blocos vem do servidor, e não do código do cliente. O selo injetado apareceu sem alteração na lógica do produto. O bloco novo foi renderizado, como indica o cabeçalho da inspeção, que passou a contar três componentes. O tipo inexistente não derrubou a tela, sendo que fora do modo de inspeção ele é ignorado, e dentro dele aparece sinalizado. O bloco sem actions foi normalizado na entrada e a tela continuou funcional.

Figura 18 – Mutações do contrato SDUI na tela de detalhe: (a) original; (b) blocos reordenados; (c) selo injetado; (d) bloco novo; (e) bloco desconhecido



Fonte: Elaborada pelos autores a partir do teste de mutação de contrato (2026).

O resultado confirma que o frontend atua como interpretador do contrato: a árvore de blocos enviada pelo servidor, e não uma sequência fixa no cliente, determina a tela. As mesmas cinco mutações foram depois produzidas pelo próprio servidor: no modo de apresentação, o cliente apenas pede uma mutação por um cabeçalho da requisição, e o BFF altera a árvore que ele mesmo montou antes de enviá-la. Um segundo teste no navegador, sem interceptar a rede, confirmou que a tela obedece sem recarregar (Figura 19). Resta um limite: as mutações foram programadas no servidor para a demonstração; o teste mostra que árvores novas chegam ao cliente sem mudança nele, e não que o servidor as componha a partir de regras de negócio ainda inexistentes.

Figura 19 – Blocos da tela de detalhe reordenados pelo próprio servidor no modo de apresentação



Fonte: Elaborada pelos autores (2026).

5.2.2 Robustez do contrato: componentes desconhecidos e dados insuficientes

Uma base histórica nunca é completa: há produtos sem peso, sem avaliação ou com um único vendedor. A estratégia adotada foi omitir o que não pode ser sustentado pelos dados. Sem distância, o selo não aparece. Sem massa, a emissão

não é exibida. Acima de 100 km, o produto não recebe o selo. Sem uma alternativa melhor, nenhuma troca é sugerida. Sem avaliações, a nota é omitida.

A ausência de selo, portanto, não afirma que a entrega é poluente, apenas indica que não há dados para classificá-la. Um selo atribuído a partir de um valor padrão aumentaria artificialmente as entregas "verdes"; a omissão é menos chamativa, mas mantém a relação entre afirmação e evidência.

O mesmo cuidado aparece na apresentação dos números. Abaixo de 1 km, o modelo não distingue a distância de zero, e a interface diz "mesma região" em vez de "0 km". A jornada completa também foi percorrida por um teste automatizado apenas com o teclado, da entrada do CEP ao checkout.

A correspondência entre servidor e cliente também é verificada automaticamente, em dois níveis. Um teste compara os tipos de blocos e ações declarados no backend, no espelho em TypeScript e no registro de componentes. Um segundo compara, campo a campo e em todos os níveis, os dados de cada bloco: o backend exporta a forma de cada um a partir dos modelos Pydantic, e o frontend declara a mesma forma, que o compilador TypeScript amarra aos tipos do espelho. Um campo renomeado, removido ou acrescentado de um lado só faz a integração contínua falhar, o que foi conferido alterando o contrato de cada lado. Os testes conferem nomes de campo, não tipos de valor, e as situações avaliadas foram controladas, não exaustivas. A convivência de versões diferentes do contrato em produção não foi avaliada.

5.3 SÍNTESE DOS RESULTADOS

Os resultados quantitativos e qualitativos respondem de maneira complementar aos objetivos do trabalho. A verificação funcional confirmou as unidades, os fatores e a monotonicidade do cálculo, sem afirmar que o modelo de emissões seja uma medição operacional de transportadora.

Os resultados respondem aos quatro objetivos específicos do trabalho.

a) Arquitetura SDUI orientada a contexto: as cinco mutações do contrato foram refletidas na tela sem novo build, tanto injetadas na resposta quanto produzidas pelo próprio servidor, e componentes desconhecidos ou incompletos não interromperam a jornada. O servidor determina a composição; o cliente a interpreta.

b) Motor de pegada de carbono baseado em atividade: o motor passou nos testes de unidade, conversão e monotonicidade. A distância corrigida pela circuidade erra 8,7% em relação à rota do OSRM e escolhe o mesmo vendedor mais próximo em 86,5% das decisões, com superestimação de 5% a 10% nas distâncias longas. Variar os parâmetros do motor altera a emissão absoluta, mas a parcela evitada fica entre 9,5% e 10,0%.

c) Personalização integrada aos dados logísticos: a página inicial adapta-se à região do comprador em 12 das 27 UFs, sem custo de desempenho. A recomendação do vendedor mais próximo reduziria em 9,8% a emissão dos itens com alternativa, equivalentes a 0,9% de todas as compras, com custo mediano de R$ 0,01 para o comprador. O alcance é limitado pela oferta de alternativas, e não pela existência de vendedores próximos, e o efeito se concentra nas regiões com mais vendedores (23,5% em SP, 3,7% na BA).

d) Desempenho e flexibilidade: a meta de 200 ms foi cumprida com 50 usuários em todos os estados e com 250 usuários quando o cache estava ativo, inclusive no código final. Com 1.000 usuários, o cache multiplicou a vazão por 2,6 e manteve a rota cacheada no limite da meta, enquanto as rotas sem cache saturaram.

Em conjunto, os resultados indicam que o artefato atende ao propósito de demonstrar uma integração entre personalização contextual, logística verde e composição server-driven. As conclusões são delimitadas pelo caráter histórico do dataset, pelos centroides de CEP, pelo modelo estimativo de emissões, pelo uso de um serviço público de roteamento e pelo ambiente controlado de carga. Essas delimitações serão retomadas no capítulo de discussão, que deverá separar o que foi demonstrado empiricamente do que permanece como hipótese, cenário ou trabalho futuro.

6 DISCUSSÃO E CONSIDERAÇÕES FINAIS

Este capítulo interpreta os resultados do Capítulo 5 à luz do problema e dos objetivos definidos na Introdução. Primeiro, discutem-se as relações de compromisso assumidas no projeto e a aderência do artefato aos objetivos e aos critérios de aceitação. Em seguida, são apresentadas as limitações e as ameaças à validade dos resultados, a conclusão do trabalho e os trabalhos futuros.

6.1 DISCUSSÃO

6.1.1 Relação de compromisso no projeto

Toda decisão relevante do projeto trocou um benefício por um custo declarado. As principais estão reunidas a seguir.

Contrato tipado em dois lados. A Server-Driven UI concentra no servidor a decisão sobre o que aparece na tela, mas exige que o contrato exista duas vezes: em Pydantic, no backend, e em TypeScript, no frontend. A duplicação é o preço do desacoplamento. O projeto não a eliminou; tornou a divergência detectável, com testes que comparam tipos de blocos, tipos de ações e campos de cada bloco em todos os níveis. Um contrato genérico, com campos livres, evitaria a duplicação, mas deixaria o erro aparecer só na tela do usuário. O custo do contrato na rede é pequeno: as telas medidas ocupam de 3 a 13 KB de JSON, ou de 1,5 a 4,3 KB comprimidos.

Proximidade em vez de rota. A distância é calculada em linha reta entre centroides de prefixos de CEP e corrigida por um fator de circuidade. Uma rota real por requisição seria mais precisa, mas acrescentaria latência e uma dependência externa ao caminho de cada tela. A comparação com o OSRM mostrou que a aproximação erra 8,7% e preserva a escolha do vendedor mais próximo em 86,5% das decisões. Para comparar vendedores, que é o uso do sistema, a precisão obtida é suficiente; para informar a emissão de uma entrega específica, não é, e por isso a interface apresenta os valores como estimativas.

Selo por distância, e não por emissão. O selo de logística verde usa um limite de 100 km, simples de comunicar e de verificar pelo usuário. A escolha tem um efeito

colateral: um produto pesado a 99 km recebe o selo, e um produto leve a 101 km, não. Um selo por emissão relativa seria mais fiel ao impacto, mas mais difícil de explicar em uma etiqueta. O trabalho optou pela comunicação clara e exibe a emissão estimada ao lado do selo, para que o usuário compare.

Proximidade antes do preço. A regra de escolha da oferta prioriza a menor distância e só usa o preço para desempatar ofertas praticamente equidistantes (até 1% da menor distância). O comparador de origens mostrou um caso que essa regra decide contra o comprador: para o mesmo produto e o mesmo CEP, a oferta escolhida custa R$ 337,69 a 22 km, enquanto outra, a 28 km, custa R$ 130,00 e emite apenas 0,4 g a mais. A regra é coerente com a proposta do trabalho, mas não é a única defensável. Sobre as compras reais, o desempate por preço já muda a escolha em 12,7% dos itens, reduz o preço em R$ 6,11 na mediana e mantém o CO_2 evitado em 9,8%. Ampliar a tolerância, por exemplo tratando como empate diferenças de emissão menores que 1 g, reduziria casos como o citado, ao custo de parte do benefício ambiental. Como a diferença de preço é visível no comparador, a decisão final continua com o usuário. O mesmo vale para a sugestão de troca no checkout, que propõe outro produto da mesma categoria vindo de mais perto: ela exibe o preço do substituto, mas não o limita. No caso usado na demonstração, a troca evitaria 4,58 g de CO_2 ao custo de R$ 142,00 a mais, uma diferença que a interface deixa visível, mas que a regra não pondera.

Eficiência e equidade regional. A recomendação de proximidade rende mais onde já há mais vendedores. Nos itens com alternativa, ela evitaria 23,5% da emissão em SP e 3,7% na BA, e SP responde por 42% de todo o CO_2 evitado. No outro extremo, nenhum comprador do PA tinha vendedor a menos de 100 km. A regra, por construção, beneficia as regiões já bem servidas; reduzir a emissão das compras do Norte e de parte do Nordeste dependeria de abrir oferta nessas regiões, e não apenas de recomendar melhor.

Personalização regional, e não individual. Sem autenticação nem histórico, o sistema conhece apenas o CEP do comprador. A vitrine se adapta à UF, pela categoria com maior lift regional, e não à pessoa. Essa escolha evitou um histórico sintético e não teve custo de desempenho, mas limita o alcance da personalização: 15 das 27 UFs não têm volume suficiente e recebem a vitrine geral.

Dados estáticos em memória e cache de respostas. Os centroides de CEP e o contexto regional ficam em memória, e as telas da página inicial ficam no Redis por uma hora. As duas decisões reduzem o custo por requisição, e o Redis foi decisivo a partir de 250 usuários. Em troca, o sistema passa a depender de um procedimento operacional: recarregar a base exige reiniciar a API e limpar o cache, ou a tela mostrará dados antigos. Em um catálogo estático como o da Olist, o custo é baixo; em um catálogo vivo, exigiria invalidação por evento.

Amostra na aplicação, base completa nas análises. A aplicação usa uma amostra de 60.636 itens, e as análises de logística verde usam a base completa. A separação mantém a aplicação leve e as análises representativas, mas exige que o leitor não confunda os dois universos, razão pela qual o Capítulo 5 os apresenta separadamente.

6.1.2 Aderência aos objetivos e critérios de aceitação

O Quadro 1 relaciona cada objetivo específico à evidência obtida.

Quadro 1 – Aderência aos objetivos específicos

Objetivo específico	Evidência	Situação
a) Arquitetura SDUI orientada a contexto, sem depender de atualizações do front-end	Doze componentes descritos pelo servidor; cinco mutações do contrato refletidas na tela sem nova compilação, injetadas na resposta e produzidas pelo próprio servidor; componentes desconhecidos ignorados sem interromper a tela	Atendido
b) Engine de pegada de carbono baseada em atividade	Motor em cadeia (circuidade, última milha por van, transferência por caminhão pesado, massa cobrável) com fatores do GLEC; testes de unidade, conversão e monotonicidade; erro de distância de 8,7% contra o OSRM; parcela evitada estável (9,5% a 10,0%) sob variação dos parâmetros	Atendido como estimativa; sem validação contra dados de transportadora
c) Personalização integrada a dados logísticos, identificando compras locais	Vitrine regional em 12 das 27 UFs; selo, distância e emissão por oferta; comparador de origens; sugestão de troca no checkout; potencial de 9,8% de redução nos itens com alternativa (0,9% do total), concentrado nas regiões com mais vendedores; comparador que apresenta a redução relativa prevista na Seção 3.2	Atendido no nível regional; efeito sobre o comportamento do usuário não medido
d) Avaliar desempenho e flexibilidade	Protocolo de 27 ensaios em três cargas e três estados de cache; controle repetido no código final; mutação do contrato	Atendido, com a meta de 200 ms cumprida até 250 usuários com cache e não cumprida no p95 agregado com 1.000 usuários

Fonte: Elaborado pelos autores (2026).

Os quatro grupos de critérios de aceitação definidos na Seção 4.1.2 foram verificados. Os cálculos de distância, massa e emissão passaram nos testes com entradas controladas. A ordenação de ofertas, remessas e alternativas é coberta por testes de unidade e pela regra única de escolha de oferta, usada pela vitrine, pelo detalhe e pelo checkout. A geração e a interpretação do contrato são verificadas nos dois lados e por testes de navegador. A jornada completa é percorrida por testes automatizados no navegador, e o comportamento sob carga foi medido pelo protocolo da metodologia.

O critério de desempenho merece uma leitura separada. A meta de tempo de resposta inferior a 200 ms foi definida como experimental, e os resultados mostram que ela depende do nível de concorrência. Com 50 usuários, foi cumprida mesmo sem o cache de respostas; com 250, só com ele; com 1.000, a rota em cache ficou no limite da meta, mas as telas de detalhe e de checkout, que não passam pelo cache, formaram fila e elevaram o p95 agregado para cerca de 3 s. O resultado não invalida a arquitetura, mas indica o próximo gargalo: estender o cache a essas telas ou distribuir a carga entre mais processos.

6.2 LIMITAÇÕES E AMEAÇAS À VALIDADE

As limitações são agrupadas pelo tipo de validade que afetam.

Validade de construto. A emissão de CO_2 é uma estimativa de modelo, e não uma medição. Os fatores são médias internacionais do GLEC, e não fatores da frota brasileira, e a Olist não informa veículo, rota, ocupação nem consumo. O selo mede distância, e não emissão. As modalidades expressa e verde são cenários declarados sobre a linha de base, sustentados na direção, mas não no valor, pela literatura. No desempenho, a ferramenta de carga mede a resposta completa, um limite superior do tempo até o primeiro byte.

Validade interna. API, banco, cache e gerador de carga dividiram o mesmo computador, e o gerador saturou em um dos ensaios, que foi refeito. Treze ensaios foram descartados porque o computador funcionou na bateria. Com três repetições, a dispersão na carga máxima é alta. O protocolo completo foi executado antes das últimas mudanças de regra; o controle no código final cobriu apenas o cenário de 250 usuários.

Validade externa. A base é histórica (2016 a 2018): a presença de um vendedor indica que a venda ocorreu, e não que a oferta estaria disponível hoje. As distâncias são calculadas entre centroides de prefixos de CEP, e não entre endereços, e o OSRM representa a rota mais rápida de carro, e não a da transportadora. A amostra da aplicação favorece os produtos mais vendidos e não representa as proporções do catálogo inteiro. Sem nome de produto na base, a interface identifica cada item pela categoria, o que limita a avaliação da experiência de compra. Os

resultados de carga comparam configurações entre si em uma máquina, e não representam um serviço distribuído.

Validade de conclusão. O CO_2 evitado é contrafactual: supõe que todos os compradores seguiriam a recomendação e que a oferta mais próxima estaria disponível. A concordância de 86,5% entre linha reta e rota vem de 37 decisões, com intervalo de confiança de 95% entre 72% e 94%. Os limiares da personalização regional protegem contra ruído, mas não constituem um teste de significância.

Validação com pessoas. O trabalho não avaliou se os usuários compreendem o selo e a estimativa de emissão, nem se mudariam de vendedor por causa deles. As conclusões sobre o potencial de redução dependem dessa hipótese.

6.3 CONCLUSÃO

O trabalho partiu de dois problemas do comércio eletrônico: a rigidez de interfaces que dependem de novas versões do cliente para mudar e a falta de informação ambiental no momento da compra. Para tratá-los em conjunto, foi desenvolvido um marketplace sobre o conjunto de dados público da Olist, no qual o servidor compõe a interface por meio de uma engine de Server-Driven UI e cada oferta carrega distância, selo e emissão estimada.

O objetivo geral foi atingido no escopo de um protótipo. A arquitetura mostrou que a composição da tela pode mudar no servidor sem nova compilação do cliente e que o cliente tolera componentes desconhecidos e dados incompletos. O motor de emissão, baseado em atividade, produziu estimativas internamente consistentes e uma aproximação de distância validada contra rotas reais. A personalização contextual foi integrada à logística verde no nível regional, e a análise da base completa mostrou que a recomendação de vendedores mais próximos tem potencial de reduzir 9,8% da emissão onde há alternativas, efeito que se dilui para 0,9% no conjunto das compras. Sob carga, o cache de respostas foi a decisão que manteve a aplicação dentro da meta até 250 usuários simultâneos.

Os resultados também delimitam a contribuição. O maior obstáculo à logística verde observado na base não foi a falta de vendedores próximos, presentes para 94,7% dos compradores, mas a falta de alternativas para o mesmo produto, que ocorre em 12,8% dos itens. Uma interface pode tornar a escolha local visível e

comparável, mas o efeito depende de haver o que escolher e de o comprador decidir a partir da informação apresentada, e hoje se concentra nas regiões que já contam com mais vendedores. Nesse sentido, o artefato demonstra a viabilidade técnica da abordagem e deixa a medição do efeito sobre o comportamento como o passo seguinte.

6.4 TRABALHOS FUTUROS

Os itens a seguir estão ordenados do que mais afetaria as conclusões para o que menos afetaria.

a) **Medir o efeito da recomendação com usuários**, por teste A/B ou estudo de preferência, para substituir o CO_2 evitado contrafactual por um efeito observado.

b) **Personalização por pessoa**, com autenticação e pedidos persistidos, separando a base de referência da Olist da base da aplicação; o contexto regional passaria a ser o ponto de partida para quem ainda não tem histórico.

c) **Significância do contexto regional**, por *bootstrap* da participação de cada categoria por UF, reavaliando os limiares de volume e de lift.

d) **Selo por emissão relativa**, comparando a emissão da oferta com a mediana da categoria, para corrigir o caso do produto pesado próximo e do leve distante.

e) **Regra de oferta com tolerância por emissão**, tratando como equivalentes ofertas cuja diferença de CO_2 seja menor que a incerteza do modelo, e medindo o custo ambiental dessa ampliação com a mesma análise do Capítulo 5; na sugestão de troca do checkout, um limite de diferença de preço.

f) **Distância rodoviária pré-calculada**, com uma matriz do OSRM entre os prefixos de vendedores e de compradores, consultada em memória, ou um fator de circuidade por faixa de distância estimado com amostra maior.

g) **Fatores de emissão da frota brasileira**, por tipo de veículo, quando houver inventário nacional com o mesmo recorte do GLEC.

h) **Escala horizontal com cache**, repetindo o protocolo com vários processos e cache ligado, com o gerador de carga em outra máquina, e estendendo o cache às telas de detalhe e de checkout.

i) **Lado do vendedor**, atendendo a persona do microempreendedor com um painel da pegada das próprias entregas e a sugestão de onde manter estoque.

j) **Versionamento de componentes**, com roteamento entre versões quando houver duas versões vivas do mesmo bloco e clientes antigos a atender.

k) **Instrumentação permanente**, com contagem de consultas e tempo por requisição atrás de uma opção de diagnóstico, para que regressões de desempenho como a encontrada durante o desenvolvimento sejam detectadas por teste.

REFERÊNCIAS

CANO, Jose Alejandro; LONDOÑO-PINEDA, Abraham; OVALLE RODAS, C. Sustainable Logistics for E-Commerce: A Literature Review and Bibliometric Analysis. **Sustainability**, v. 14, n. 19, art. 12247, 27 set. 2022. DOI: 10.3390/su141912247.

CHERIPURATHU, Kurian George; KULKARNI, Sanjeev. Integrating Microservices and Microfrontends: A Comprehensive Literature Review on Architecture, Design Patterns, and Implementation Challenges. **Journal of Scientific Research and Technology**, p. 1-12, 2 jul. 2024. DOI: 10.61808/jsrt115.

EDRONE. **Dados do e-commerce no Brasil**: estatísticas e tendências. [S. l.], [202-?]. Disponível em: https://edrone.me/br/blog/dados-ecommerce-brasil. Acesso em: 14 abr. 2026.

ISO. **ISO 14083:2023**: Greenhouse gases — Quantification and reporting of greenhouse gas emissions arising from operations of transport chains. Geneva: International Organization for Standardization, 2023. 108 p.

LEE, K.; CHAE, J. Estimation of travel cost between geographic coordinates using artificial neural network: potential application in vehicle routing problems. **ISPRS International Journal of Geo-Information**, Basel, v. 12, n. 2, p. 57, 2023. DOI: 10.3390/ijgi12020057.

LOCUST. **Locust**: an open source load testing tool. Versão 2.46. [S. l.]: Locust.io, 2024. Disponível em: https://docs.locust.io. Acesso em: 9 set. 2026.

MUNDIM, Arianne Alves da Silva. **Roteamento de Estoque e Transporte de Carga Multimodal**: Otimização Bi-objetivo com Considerações Ambientais e Econômicas. 2024. 114 p. Tese (Doutorado em Ciências de Computação e Matemática Computacional) – Universidade de São Paulo, São Paulo, 2024.

NAWAZ, Asif et al. Context-aware Frameworks and Architectures — A Comprehensive Survey. In: INTERNATIONAL CONFERENCE ON ADVANCED SCIENCE AND ENGINEERING TECHNOLOGIES (ASET), 2022. **Proceedings** [...]. IEEE, 21 fev. 2022. DOI: 10.1109/ASET53988.2022.9735024.

OLIST. **Brazilian E-Commerce Public Dataset by Olist**. Kaggle, 2024. Disponível em: https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce. Acesso em: 28 mar. 2026.

OLIST. **Como calcular o frete dos produtos para envio pelos Correios?** São Paulo: Olist, 20 set. 2022. Disponível em: https://olist.com/blog/pt/gestao-empresarial/operacao-e-logistica/como-calcular-o-frete-dos-produtos-na-loja-virtual/. Acesso em: 13 out. 2026.

OLIVEIRA, M. **O panorama do e-commerce no Brasil**: tendências e desafios. São Paulo: Varejo Digital, 2022. 41 p.

ORGANIZAÇÃO DAS NAÇÕES UNIDAS NO BRASIL. **As Nações Unidas no Brasil**. Brasília: ONU Brasil, 2024. Disponível em: https://brasil.un.org/pt-br. Acesso em: 28 mar. 2026.

PEREIRA, T. **Logística verde e a economia de baixo carbono**. Manaus: EcoLog, 2024. 29 p.

PERLIN, Rodrigo et al. An Approach to Follow Microservices Principles in Frontend. In: IEEE INTERNATIONAL CONFERENCE ON ADVANCED INFORMATION TECHNOLOGIES (AICT), 2023. **Proceedings** [...]. IEEE, 18 out. 2023. DOI: 10.1109/aict59525.2023.10313208.

PROGRAMA BRASILEIRO GHG PROTOCOL. **Ferramenta de cálculo**. São Paulo: FGVces, 2026. Disponível em: https://ghgprotocolbrasil.com.br/ferramentas. Acesso em: 17 maio 2026.

RAMÍREZ, Sebastián. **FastAPI**: high performance web framework for building APIs with Python. Versão 0.141. [S. l.]: Tiangolo, 2024. Disponível em: https://fastapi.tiangolo.com. Acesso em: 9 set. 2026.

RAPPL, Florian. **The Art of Micro Frontends**: Build websites using compositional UIs that grow naturally as your application scales. 2. ed. Birmingham: Packt Publishing, 2024. 356 p.

RATTANUKUL, Pattarakrit et al. Microusity: A testing tool for Backends for Frontends (BFF) Microservice Systems. In: INTERNATIONAL CONFERENCE ON PROGRAM COMPREHENSION (ICPC), 2023. **Proceedings** [...]. IEEE, maio 2023. DOI: 10.1109/icpc58990.2023.00021.

REDIS. **Redis documentation**: in-memory data store and caching strategies. [S. l.]: Redis Ltd., 2024. Disponível em: https://redis.io/docs. Acesso em: 9 set. 2026.

ROCHA, Hugo Filipe Oliveira. **Practical Event-Driven Microservices Architecture**: Building Sustainable and Highly Scalable Event-Driven Microservices. New York: Apress, 2022. 469 p.

SILVA, Sérgio Ferrigo da. **Inovação sustentável integrando Internet das coisas à logística 5.0**: armazéns inteligentes como infraestruturas de serviço de apoio à sociedade. 2023. 152 p. Tese (Doutorado em Administração) – Universidade de Caxias do Sul, Caxias do Sul, 2023.

SIMÃO, Angelo Guimarães; SALLES, Dayane Martins; FERNANDES, Valdir. Plataformas digitais de indicadores dos ODS: origem, abrangência e transparência. **Revista Tecnologia e Sociedade**, v. 20, n. 59, 15 abr. 2024. DOI: 10.3895/rts.v20n59.17828.

SMART FREIGHT CENTRE. **Global Logistics Emissions Council Framework for Logistics Emissions Accounting and Reporting**. Version 2.0. Amsterdam: Smart Freight Centre, 2019. Atualizado em 2022. Disponível em: https://www.smartfreightcentre.org/en/glec-framework/. Acesso em: 9 set. 2026.

SMART FREIGHT CENTRE. **Global Logistics Emissions Council Framework for Logistics Emissions Accounting and Reporting (GLEC Framework)**. Versão 3.0. Amsterdam: Smart Freight Centre, 2023. 164 p. Disponível em: https://www.smartfreightcentre.org/en/glec-framework/. Acesso em: 9 set. 2026.

TRANSVIAS. **Pegada de carbono no transporte de cargas**: como reduzir impactos ambientais. [S. l.], 6 out. 2025. Disponível em: https://noticias.transvias.com.br/2025/10/06/pegada-carbono-transporte-cargas/. Acesso em: 16 abr. 2026.

WORLD ECONOMIC FORUM. **Transforming Urban Logistics**. [S. l.]: World Economic Forum, 2024. Disponível em: https://www.weforum.org/publications/transforming-urban-logistics-a-path-to-sustainable-and-efficient-last-mile-delivery-in-cities/. Acesso em: 13 out. 2026.

ZUBAIDAH, Tantri Risda et al. Sustainable Business of E-Commerce in the Smart Economy: A Systematic Review of Roles and Challenges for Future Research. In: INTERNATIONAL CONFERENCE ON INFORMATION SYSTEMS AND SOFTWARE ENGINEERING (ICISS), 2024. **Proceedings** [...]. IEEE, 2024. DOI: 10.1109/iciss62896.2024.10751282.